

INTERNSHIP REPORT

A report submitted in fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

SEHAAN UR RAHMAN

Regd. No.: 23BCE11197

Under Supervision of

Mrs. Pavithra Rathinavel

Assistant Professor, Department of CSE, VIT Bhopal University

and

MPOnline Limited, Bhopal

(Duration: 20th May, 2026 to 31st July, 2026)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

October 2026

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY



CERTIFICATE

This is to certify that the “Internship Report” submitted by **SEHAAN UR RAHMAN** (Regd. No.: 23BCE11197) is the work done by the student during the internship programmes undertaken at **MPOnline Limited, Bhopal**, from 20th May, 2026 to 31st July, 2026, and is submitted in partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (Session 2023 – 2027) at VIT Bhopal University.

The internships reported herein comprise the **Advanced Software Engineering and Development Internship** (programme code SE3) and the **AI/ML Internship Program** (programme code AI3), both carried out at MPOnline Limited, Bhopal. The work was carried out under the industry supervision of **Mr. Rupesh Phalne**, Industry Mentor, MPOnline Limited, and under the academic supervision of **Mrs. Pavithra Rathinavel**, Department of Computer Science and Engineering, VIT Bhopal University. The prescribed period of seventy-three (73) days has been completed to the satisfaction of the Department, and the reports of the work submitted are the original work of the candidate.

College Internship Guide

Signature:

Manikandan Jayapal Associate Professor
VIT Bhopal University

CERTIFICATION

Certification of Internship 1 — Advanced Software Engineering and Development

The certification reproduced below is the **Certificate of Completion** issued by MPOne Limited, Bhopal, on successful completion of the Advanced Software Engineering and Development Internship (programme code SE3). A reproduction of the certificate appears as **Figure 1**, its particulars are recorded in the reference block that follows, and the technical scope of the training is set out in the Weekly Overview of Internship Activities and in Chapter 2 of this report.



Figure 1: Certificate of Completion issued by MPOne Limited for the Advanced Software Engineering and Development Internship (programme code SE3)

Certificate ID	MPO/INT/26-27/07/2412/26052416
Internship	Advanced Software Engineering and Development Internship (SE3)
Issuing Authority	MPOne Limited, Bhopal
Certificate Issued By	Gagan Soni, Chief Technical Officer, MPOne Limited
Duration of Training	20th May, 2026 to 31st July, 2026 (73 days)
Industry Mentor	Mr. Rupesh Phalne
Academic Institution	VIT Bhopal University

Certification of Internship 2 — Artificial Intelligence and Machine Learning

The certification reproduced below is the **Certificate of Completion** issued by MPOnline Limited, Bhopal, on successful completion of the AI/ML Internship Program (programme code AI3). A reproduction of the certificate appears as **Figure 2**, its particulars are recorded in the reference block that follows, and the technical scope of the training is set out in the Weekly Overview of Internship Activities and in Chapter 3 of this report.



Figure 2: Certificate of Completion issued by MPOnline Limited for the AI/ML Internship Program (programme code AI3)

Certificate ID	MPO/INT/26-27/07/1117/26051121
Internship	AI/ML Internship Program (AI3), Batch 1A
Issuing Authority	MPOnline Limited, Bhopal
Certificate Issued By	Gagan Soni, Chief Technical Officer, MPOnline Limited
Duration of Training	20th May, 2026 to 31st July, 2026 (73 days)
Programme Instructor	Mr. Nishant Shrivastava
Academic Institution	VIT Bhopal University

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to **Mr. Rupesh Phalne**, Industry Mentor at MPOnline Limited, Bhopal, for granting me the opportunity to carry out the Advanced Software Engineering and Development Internship within the organisation, and for the patient guidance, the review of each implementation and the candid feedback extended over the seventy-three (73) days of the internship. I am equally grateful to **Mr. Nishant Shrivastava**, instructor of the AI/ML Internship Program (Batch 1A), whose carefully prepared notes, clear explanations and prompt guidance made it possible to move from the fundamentals of Python to deployed machine learning applications within a single programme; and to my fellow participants of Batch 1A, whose questions and discussions created an open and enjoyable learning environment.

I am deeply grateful to the management of **MPOnline Limited, Bhopal**, for admitting me to its engineering environment, for extending the development workstations and the licensed toolchain on which the technical work depended, and for permitting the work to be retained as a departmental reference.

I am greatly indebted to **Mrs. Pavithra Rathinavel**, my College Internship Guide in the Department of Computer Science and Engineering, VIT Bhopal University, whose support and advice were instrumental in securing these placements, in completing both programmes on schedule and in preparing this report to the standard the Department requires. I am highly obliged to **Dr. Pushpinder Singh Patheja**, Professor and Dean, School of Computing Science and Engineering (SCOPE), for the facilities and encouragement provided for the conduct of these internships, and I thank **Dr. Ravi Verma**, Programme Chair, CSE-Core, SCOPE, for the constructive suggestions offered throughout the programme.

I am extremely grateful to the faculty of the Department, to my classmates and to my family, who encouraged me towards the successful completion of these internships and of this report.

SEHAAN UR RAHMAN

(Regd. No.: 23BCE11197)

EXECUTIVE SUMMARY / ABSTRACT

The Company. MPOnline Limited is the e-governance service delivery company of the Government of Madhya Pradesh. Incorporated in 2006 as a joint venture between the Government of Madhya Pradesh and Tata Consultancy Services and headquartered in Bhopal, it builds and operates the citizen services portal hosted at www.mponline.gov.in, through which Government departments deliver online applications and payments, examination and recruitment processes, admission counselling, licences, certificates and welfare schemes, supported by a statewide network of authorised kiosks. The organisation is ISO certified and appraised at CMMI Maturity Level 3, and it conducts structured technology internships within its engineering environment under designated industry mentors and instructors.

The Problem or Opportunity. An e-governance service delivery organisation depends entirely upon maintainable, auditable software and correctly designed relational data structures, because every citizen-facing transaction traverses these systems and every record becomes subject to statutory scrutiny; the same organisation accumulates large volumes of structured data from which machine learning can derive forecasting, classification and question-answering services. Two gaps were therefore addressed: the gap between academic programming and industrial software construction, in which code clarity, object-oriented design, presentation quality and schema integrity together determine maintainability; and the gap between training a model and delivering one, since a model must be evaluated honestly, packaged as a service and deployed where real users can reach it.

Methodology. The two programmes ran concurrently from 20th May, 2026 to 31st July, 2026, a span of seventy-three (73) calendar days, under the academic supervision of Mrs. Pavithra Rathinavel of the Department of Computer Science and Engineering, VIT Bhopal University. The Advanced Software Engineering and Development Internship (SE3) was supervised by Mr. Rupesh Phalne as a progressively scaffolded apprenticeship across eleven weekly cycles and sixty-three (63) working days from Monday to Saturday, in which each day paired a theory briefing with supervised laboratory implementation. The AI/ML Internship Program (AI3, Batch 1A) was conducted by Mr. Nishant Shrivastava in weekday sessions from 1:00 pm to 3:00 pm, combining concept sessions, guided implementation in Python and continuous

assessment through ten graded assignments submitted as version-controlled repositories.

Key Parts of the Report, Findings and Solutions. Chapter 1 introduces the domain, the organisation and the two programmes; Chapter 2 discusses Internship 1 module by module; Chapter 3 discusses Internship 2; and Chapter 4 states the conclusions. The principal artefact of Internship 1 is a C# payroll hierarchy in which an Employee base class is extended by Programmer, AssistantProfessor, AssociateProfessor and Professor, each computing Dearness Allowance at ninety-seven per cent and House Rent Allowance at ten per cent of Basic Pay and deducting Provident Fund at twelve per cent and Staff Club Fund at one-tenth of one per cent, before emitting a pay slip; the chapter also records a polymorphic weekly payroll engine, an accessible HTML5 asset set, ten CSS exercises, a normalised SQL Server schema and an ASP.NET MVC pay-slip portal. The main project of Internship 2 is a Car Price Prediction System that estimates used-vehicle resale prices with a Random Forest model (R^2 of 0.958 on unseen data and 0.920 by cross-validation), served through a Flask application deployed on Render and packaged with Docker. Its central finding is methodological: a high score does not by itself establish a good model, because duplicate records, target leakage, class imbalance and test data reused for validation each produced apparently excellent results that did not survive scrutiny.

Benefits to the Company and the Institution. For MPOnline Limited, the internships produced a documented body of reference implementations — an object-oriented payroll model, a normalised relational schema with verified query results, a presentation-layer asset set, an integrated web portal and a collection of reproducible machine learning projects — accompanied by an explicit record of the pitfalls encountered and the corrections applied, all of which can serve as baselines when new code is examined and are directly relevant to e-governance uses such as service-demand forecasting and document question-answering for help desks. For VIT Bhopal University, the report documents industry-oriented training that complements the B.Tech CSE curriculum.

Keywords: MPOnline Limited; C# and .NET; HTML5 and CSS; SQL Server; ASP.NET MVC; Python; machine learning; deep learning; deployment.

INDEX

S. No.	Contents	Page No.
1.	Certificate	ii
2.	Certification — Certificate of Completion issued by MPOne Limited for Internship 1 (Advanced Software Engineering and Development, SE3)	iii
3.	Certification — Certificate of Completion issued by MPOne Limited for Internship 2 (AI/ML Internship Program, AI3)	iv
4.	Acknowledgement	v
5.	Executive Summary / Abstract	vi
6.	Index	viii
7.	Learning Objectives / Internship Objectives	ix
8.	Weekly Overview of Internship Activities	x
8.1	Part A — Internship 1 (SE3): eleven weekly cycles	xii
8.1.1	Week 1: Induction, Organisational Context and Development Environment Setup	xii
8.1.2	Week 2: C# Language Fundamentals and Console Programming	xiii
8.1.3	Week 3: Class Design, Encapsulation and Inheritance Foundations	xiv
8.1.4	Week 4: The Employee Payroll Hierarchy and Polymorphic Payroll	xv
8.1.5	Week 5: HTML5 Document Structure, Forms and Accessibility	xvi
8.1.6	Week 6: CSS Application Mechanisms, Selectors and Background Styling	xvii
8.1.7	Week 7: CSS Pseudo-Elements, Interactive Components and Responsive Layout	xviii
8.1.8	Week 8: RDBMS Fundamentals, SQL Server and Data Definition Languages	xix
8.1.9	Week 9: Data Manipulation, Query Operators and Normalisation	xx
8.1.10	Week 10: Enterprise Web Applications with ASP.NET MVC	xxi
8.1.11	Week 11: Consolidation, Documentation, Report Submission and Viva Preparation	xxii
8.2	Part B — Internship 2 (AI3): eleven weekly cycles	xxiv
8.2.1	Week 1: Orientation and Python Fundamentals	xxiv
8.2.2	Week 2: Python Control Flow, Functions, NumPy and Pandas	xxv
8.2.3	Week 3: Data Handling, Mathematics for ML, EDA and Feature Engineering	xxvi
8.2.4	Week 4: Supervised Machine Learning using Linear Regression	xxvii
8.2.5	Week 5: Polynomial Regression, Classification and K-Nearest Neighbors	xxviii
8.2.6	Week 6: Tree-Based Models, SVM and Unsupervised Learning	xxix
8.2.7	Week 7: Model Deployment, Docker and Introduction to Reinforcement Learning	xxx
8.2.8	Week 8: MLOps, Industry Practices and Capstone Projects	xxxi
8.2.9	Week 9: Deep Learning and Natural Language Processing	xxxii
8.2.10	Week 10: Prompt Engineering, Advanced RAG and Assignments 1–4	xxxiii
8.2.11	Week 11: Assignments 5–10, Portfolio and Completion	xxxiv
9.	Chapter 1: Introduction	1
1.1	The Domain: Software Engineering and Applied Machine Learning in Enterprise Systems	1
1.2	About the Host Organisation: MPOne Limited	1
1.3	The Two Internship Programmes: Scope, Structure and Responsibilities	2
1.4	Tools, Frameworks and Technology Stack	3
1.5	Organisation of the Report	3
10.	Chapter 2: Internship 1 Discussion — Advanced Software Engineering and Development	4
2.1	How the Internship 1 Objectives Were Achieved	4
2.2	Technical Learning During the Internship	5
2.3	Principal Deliverables and Verified Results	10
2.4	Results and Observations	11
2.5	Skills Acquired During the Internship	12
2.6	Challenges Encountered and How They Were Resolved	14
11.	Chapter 3: Internship 2 Discussion — Artificial Intelligence and Machine Learning	16

S. No.	Contents	Page No.
3.1	How the Internship 2 Objectives Were Achieved	16
3.2	Technical Learning During the Internship	17
3.3	Main Project: Car Price Prediction System	21
3.4	The Ten Graded Assignments	25
3.5	Results, Observations and Corrected Figures	26
3.6	Skills Acquired During the Internship	28
3.7	Challenges Faced and How They Were Overcome	29
12.	Chapter 4: Conclusion	31
13.	Chapter 5: Bibliography	32
5.1	Books and Research Papers	32
5.2	Standards, Specifications and Vendor Documentation	32
5.3	Data Sets and Application Programming Interfaces	33

LEARNING OBJECTIVES / INTERNSHIP OBJECTIVES

The objectives below were formulated before commencement with the Department of Computer Science and Engineering, VIT Bhopal University, and with the mentor and instructor designated by MPOnline Limited, and were recorded in the internship logbook at induction.

Internship 1 — Advanced Software Engineering and Development (SE3)

1. To acquire working command of C# and the .NET object-oriented model.
2. To design classes that encapsulate state behind access modifiers.
3. To build inheritance hierarchies with overriding and polymorphism.
4. To use abstract classes and interfaces as implementation contracts.
5. To model a payroll hierarchy with DA at 97%, HRA at 10% and PF at 12% of Basic Pay.
6. To implement a polymorphic weekly payroll engine over four employee types.
7. To construct accessible HTML5 documents, labelled forms and scoped tables.
8. To apply inline, internal and external styles and explain specificity.
9. To style content with pseudo-classes and build interactive components.
10. To design a SQL Server schema normalised to Third Normal Form.
11. To build an ASP.NET MVC application with validation and Razor views.
12. To practise version control, documentation and delivery discipline.

Internship 2 — Artificial Intelligence and Machine Learning (AI3)

1. To clean, analyse and visualise data with NumPy, Pandas and Seaborn.
2. To build, evaluate and compare supervised models with scikit-learn.
3. To apply K-Means clustering and Principal Component Analysis.
4. To design and train neural networks and to control overfitting.
5. To understand prompt engineering and Retrieval-Augmented Generation.
6. To build recommendation systems and train reinforcement learning agents.
7. To serve and deploy trained models with Flask, Docker and Render.

8. To follow MLOps practice in packaging and continuous integration.

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

Organisation: MPOne Limited, Bhopal

Programmes: Advanced Software Engineering and Development Internship (SE3) and AI/ML Internship Program (AI3, Batch 1A)

Duration: 20th May, 2026 to 31st July, 2026

Industry Mentor (Internship 1): Mr. Rupesh Phalne, MPOne Limited

Programme Instructor (Internship 2): Mr. Nishant Shrivastava, MPOne Limited

Academic Guide: Mrs. Pavithra Rathinavel, Department of CSE, VIT Bhopal University

Programme	Schedule followed	Working days
Advanced Software Engineering and Development (SE3)	Eleven weekly cycles, Monday to Saturday, under the supervision of the industry mentor	63
AI/ML Internship Program (AI3, Batch 1A)	Eleven weekly cycles, weekday sessions from 1:00 pm to 3:00 pm, with notes, guided implementation and graded assignments	53

The matrices that follow record, for each weekly cycle of each programme and for every working day, the date, the day and the topic or module completed. Each weekly entry occupies a single page, as required by the report format. Attendance was maintained on all working days, and the activity recorded was countersigned by the industry mentor at the close of each cycle. Part A covers the eleven weekly cycles of the Advanced Software Engineering and Development Internship, and Part B covers the eleven weekly cycles of the AI/ML Internship Program, both of which ran concurrently between 20th May, 2026 and 31st July, 2026.

PART A — WEEKLY OVERVIEW OF INTERNSHIP 1 (SE3)

Advanced Software Engineering and Development Internship (programme code SE3)
— eleven weekly cycles, Monday to Saturday

The eleven cycles that follow record the work of the Advanced Software Engineering and Development Internship in the order in which it was carried out, one page for each cycle, from induction and environment provision through the C# language, class design and the employee payroll hierarchy, the HTML5 and CSS modules, the relational database and SQL Server module, and the ASP.NET MVC application, to the closing week of consolidation, documentation and viva preparation.

Week	Focus of the weekly cycle
Week 1	Induction, organisational context, source control and development environment setup
Week 2	C# language fundamentals and console programming
Week 3	Class design, encapsulation and the foundations of inheritance
Week 4	The employee payroll hierarchy and polymorphic payroll computation
Week 5	HTML5 document structure, forms and accessibility
Week 6	CSS application mechanisms, selectors and background styling
Week 7	CSS pseudo-elements, interactive components and responsive layout
Week 8	RDBMS fundamentals, SQL Server and the data definition languages
Week 9	Data manipulation, query operators and normalisation to Third Normal Form
Week 10	Enterprise web applications with ASP.NET MVC
Week 11	Consolidation, documentation, report submission and viva preparation

Each cycle closes with a summary of the module completed, the practical work submitted and the skills carried forward into the following week.

Week 1: Induction, Organisational Context and Development Environment Setup

Week	Date	Day	Name of the Topic / Module Completed
1ST WEEK	20/05/2026	Wednesday	Programme induction introduced MPOnline Limited as the e-governance service delivery company of the Government of Madhya Pradesh. Its joint-venture structure, the 2006 incorporation, the citizen services portal at www.mponline.gov.in , ISO certification and CMMI Maturity Level 3 appraisal were described. Mentor allocation, the reporting structure and the daily logbook requirement were formally established.
1ST WEEK	21/05/2026	Thursday	The e-governance service delivery context and the departmental workflows supported by the platform were examined, together with the information-security and data-handling obligations attaching to citizen service data. Workplace and IT usage policy, code of conduct and attendance were reviewed, and the escalation path to the industry mentor was established.
1ST WEEK	22/05/2026	Friday	Source control and documentation conventions with Git were established. Repository initialisation, staging and committing changes, inspection of history through git log and git diff, and topic branches for each module family were practised. A commit-message convention, a README template and a folder convention separating source, scripts and documentation were drafted.
1ST WEEK	23/05/2026	Saturday	Installation of the development toolchain: Visual Studio 2022 with the .NET SDK, Microsoft SQL Server 2019 with SQL Server Management Studio, Visual Studio Code and Git. Build verification compiled and executed a console application, confirming the MSBuild toolset, the runtime version and correct resolution of environment paths.

Summary of the week. The first week established the professional setting in which the internship was carried out. Induction introduced MPOnline Limited as the Government of Madhya Pradesh e-governance service delivery entity, together with its joint-venture structure, the certifications held and the accountability attaching to citizen-service data. Version-control discipline was introduced with Git, covering repository initialisation, topic branches, inspection of history and commit-message conventions supported by a README template. The workstation toolchain was provisioned and an initial compile-and-run cycle confirmed that Visual Studio, SQL Server and Git functioned together without further remediation.

Week 2: C# Language Fundamentals and Console Programming

Week	Date	Day	Name of the Topic / Module Completed
2ND WEEK	25/05/2026	Monday	C# language fundamentals: primitive value types, implicit and explicit conversion, the arithmetic, relational and logical operators, and if-else control flow. The first console programs printed a greeting and a name on separate lines, summed two numbers and swapped two numbers through a temporary variable.
2ND WEEK	26/05/2026	Tuesday	Console input with Console.ReadLine and conversion through int.Parse and double.Parse; multiplication of three user-entered numbers; and an arithmetic menu offering addition, subtraction, multiplication and division resolved through a switch statement. A multiplication table for an input number was generated to a defined limit.
2ND WEEK	27/05/2026	Wednesday	Repetition constructs in C#: the for loop, the while loop and the do-while loop. The average of four input numbers was computed by accumulating a double total and dividing by a count. An age-based message using an if-else-if ladder and an even-odd check with the modulus operator were implemented and verified.
2ND WEEK	28/05/2026	Thursday	Conditional logic through the if-else-if ladder: a positive-negative determination by nested if-else on the sign of the input, a leap-year check applying the divisibility rules for 4, 100 and 400, and an equality test of two integers with the == and != operators. Boolean variables and compound conditions were introduced.
2ND WEEK	29/05/2026	Friday	Nested loops, arrays and character sequences were examined. Exercise 2, the palindrome check, was implemented for the sample input MADAM by reversing the character sequence and comparing it with the original using StringComparison.OrdinalIgnoreCase, with a manual reversal through a char buffer retained for comparison.
2ND WEEK	30/05/2026	Saturday	Consolidation laboratory. The console programs of the week were recompiled and executed together, and input validation was added so that non-numeric Console.ReadLine input is rejected before conversion is attempted. A menu-driven dispatcher using switch was built to route execution, confirming operator precedence and loop termination.

Summary of the week. The second week concentrated on the imperative and decision-making foundations of C#, applied through the basic console programs of the introductory exercise set. Value types, conversion, the arithmetic and relational operators, and the if-else, switch, for, while and do-while constructs produced the greeting, summation, swapping, multiplication, calculator, multiplication-table, average, age-category, equality, sign and leap-year programs. Compilation errors arising from type mismatch and from omitted conversions were diagnosed rather than suppressed. The Saturday laboratory consolidated the set behind a menu dispatcher with defensive rejection of non-numeric input at the console boundary.

Week 3: Class Design, Encapsulation and Inheritance Foundations

Week	Date	Day	Name of the Topic / Module Completed
3RD WEEK	01/06/2026	Monday	Class design and encapsulation through Exercise 3: a class Book carrying BookId, Pages and Price as private fields, exposed through the public methods get(), show(), set() and getPrice(). Access from the entry-point class was restricted to the public surface, and the auto-implemented property was compared against an explicit backing field.
3RD WEEK	02/06/2026	Tuesday	Exercise 4: a class building exposing the members floors, area and occupants together with the method areaperperson() that divides the area by the occupant count. Two instances, house and office, were created and their state printed, demonstrating object identity, reference semantics and computed output.
3RD WEEK	03/06/2026	Wednesday	Arrays held as instance state through Exercise 5: a class Result with rollno, name and the marks of three subjects stored in an int array. The input() method populated the array, show() printed the record, total() summed the three elements and avg() returned the mean promoted to double without truncation.
3RD WEEK	04/06/2026	Thursday	Validation logic in Exercise 6: a class rectangle whose length and width fields initialise to a default value of 1, with perimeter and area computation and set()/get() methods rejecting any dimension which is not a floating-point value greater than 0.0 and less than 20.0.
3RD WEEK	05/06/2026	Friday	Consolidation of the class-design module. The modifiers public, private, protected and internal were applied across the Book, building, Result and rectangle classes; parameterised and default constructors were added; static members and method overloading were demonstrated; and pass-by-value was contrasted with pass-by-reference using an explicit ref parameter.
3RD WEEK	06/06/2026	Saturday	Inheritance fundamentals: base and derived declarations, the base keyword, constructor chaining and protected access; Exercise 9, the Food, Fruit and Apple hierarchy with a constructor in each class printing the current class name; and Exercise 15, a Clock class whose derived display() override renders both the 24-hour and AM/PM forms of the same instant.

Summary of the week. The third week advanced from procedural constructs to object-oriented class design. The Book, building, Result and rectangle classes established the conventions governing the remaining C# work: private state exposed through public methods, explicit constructor initialisation and validation at the point of assignment. The Result class confirmed the int array as instance state, and the rectangle class demonstrated rejection of dimensions outside the permitted interval. The Saturday session consolidated access modifiers, constructors, static members, method overloading and value against reference parameter passing, and the three-level Food, Fruit and Apple hierarchy established the inheritance syntax used thereafter.

Week 4: The Employee Payroll Hierarchy and Polymorphic Payroll

Week	Date	Day	Name of the Topic / Module Completed
4TH WEEK	08/06/2026	Monday	Exercise 1: construction of the Employee base class carrying Emp_name, Emp_id, Address, Mail_id and Mobile_no, extended by the derived classes Programmer, AssistantProfessor, AssociateProfessor and Professor. Base constructors were invoked from each derived class and the resulting instance state was printed.
4TH WEEK	09/06/2026	Tuesday	Payroll computation across the Employee hierarchy. Basic Pay was the base component, with Dearness Allowance at 97% of Basic Pay, House Rent Allowance at 10% of Basic Pay, Provident Fund deducted at 12% and Staff Club Fund at 0.1%. Gross and net salary were derived from those components.
4TH WEEK	10/06/2026	Wednesday	Pay-slip generation for each derived class of Exercise 1, producing a formatted statement for the Programmer, AssistantProfessor, AssociateProfessor and Professor instances displaying the component-wise entitlement, the deductions, the gross salary and the net salary. Computation invoked through an Employee-typed reference confirmed polymorphic dispatch.
4TH WEEK	11/06/2026	Thursday	Exercise 7 and Exercise 13: a SavingsAccount class with a static data member holding the annual interest rate, a private savingbalance, a monthlyInterest() method and a static modifyInterest() method, exercised with two savers holding 30,000 and 50,000 at an annual rate of 5 per cent.
4TH WEEK	12/06/2026	Friday	Exercise 11: the polymorphic weekly payroll engine. SalariedEmployee receives a fixed weekly salary, HourlyEmployee one and a half times the hourly rate beyond forty hours, CommissionEmployee a percentage of sales through CalculateCommission, and BaseSalariedCommissionEmployee a base salary plus a percentage with a ten per cent reward increment.
4TH WEEK	13/06/2026	Saturday	Exercises 10 and 14: a Worker base class extended by Employee and Manager, with a Project class carrying a name and completion percentage, dynamic allocation and constructor initialisation; and the RegisterForExams interface implemented by Student and Employee behind a test method taking an interface-typed parameter.

Summary of the week. The fourth week delivered the principal artefact of the internship. The Employee base class carrying Emp_name, Emp_id, Address, Mail_id and Mobile_no was extended by the four academic staff classes, with Dearness Allowance, House Rent Allowance, Provident Fund and Staff Club Fund supplying the payroll arithmetic at 97%, 10%, 12% and 0.1% of Basic Pay respectively. Pay-slip generation was verified against hand calculation, and centralising the formula in the base class removed duplication across the derived classes. Exercises 10, 11, 13 and 14 added a Worker hierarchy, the polymorphic weekly payroll of four employee types, the SavingsAccount static-interest model and interface-based registration.

Week 5: HTML5 Document Structure, Forms and Accessibility

Week	Date	Day	Name of the Topic / Module Completed
5TH WEEK	15/06/2026	Monday	Semantic HTML5 structure: the header, nav, main, article, section, aside and footer elements, together with the document type declaration, the character encoding and the viewport meta element. The document outline derived from the markup was examined and compared against the rendering in the browser.
5TH WEEK	16/06/2026	Tuesday	Exercise 2: an employee record form using the text, date, number, select, radio, checkbox and file input types. Every control was bound to a label, groups were delimited with fieldset and legend, and the required, minlength, maxlength, pattern, min, max and step constraints were applied and their effect on submission verified.
5TH WEEK	17/06/2026	Wednesday	Exercise 3: a class timetable table with a caption, a thead of th scope=col cells, a tbody in which each class was introduced by a th scope=row cell, and a tfoot carrying totals. Colspan and rowspan merged a laboratory block spanning several periods, and the table was checked against the specification.
5TH WEEK	18/06/2026	Thursday	Exercise 4: an audio element with controls and two source children, a video element with a poster, a preload attribute and a caption track, an iframe embedding a map, an inline svg diagram and a canvas drawn through the getContext 2D rendering context. Each element was confirmed to render as specified.
5TH WEEK	19/06/2026	Friday	Exercise 6: accessibility of the markup. A skip-to-content link was placed as the first focusable element, meaningful alt text was supplied, aria-label was applied to icon-only controls, a logical tabindex order was established, the navigation, main and contentinfo roles were assigned, and an aria-live=polite region was introduced.
5TH WEEK	20/06/2026	Saturday	Exercises 7 and 8: constraint validation with required, pattern, email, url and number inputs verified through checkValidity and setCustomValidity, and a product catalogue built with the template, details and summary elements together with a dialog opened by showModal. Keyboard operation of the dialog was verified.

Summary of the week. The fifth week opened the frontend module with the document layer. The semantic elements of HTML5 and the head declarations that govern parsing and scaling were examined against the outline produced by the browser. Exercise 2 built a fully labelled employee record form exercising the text, date, number, select, radio, checkbox and file controls together with the fieldset grouping and the constraint attributes. Exercise 3 reproduced a class timetable whose row and column headers, footer totals and merged laboratory block were checked for assistive-technology correctness, while Exercise 4 confirmed media, embedded, vector and canvas elements. Accessibility and the constraint-validation API closed the week.

Week 6: CSS Application Mechanisms, Selectors and Background Styling

Week	Date	Day	Name of the Topic / Module Completed
6TH WEEK	22/06/2026	Monday	CSS fundamentals: the anatomy of a rule comprising a selector and a declaration block, the cascade and specificity, and the inheritance of properties. Exercises 1 to 3 demonstrated inline styling through the style attribute, an internal style element and an external linked stylesheet, and the precedence differences between them were recorded.
6TH WEEK	23/06/2026	Tuesday	Exercise 4(i): a web page styled with different fonts and text styles. The font-family, font-size, font-style, font-weight, font-variant and text-decoration properties were applied to headings, body copy and emphasised spans, and the visual difference attributable to each property value was recorded against the selector that produced it.
6TH WEEK	24/06/2026	Wednesday	Exercises 4(ii) and 4(iii): background images specified for the whole page through the body element and for a single element through its class selector, using background-image, background-position, background-size and background-attachment, together with link styling through the anchor selector and hover-dependent colour changes.
6TH WEEK	25/06/2026	Thursday	Exercises 4(iv) and 4(v): working with layers through position relative, position absolute and z-index to govern the stacking order of overlapping content, and a customised cursor declared with the cursor property using a url value together with a fallback cursor keyword for formats the browser cannot render.
6TH WEEK	26/06/2026	Friday	Exercise 5: a web page carrying styles defined per selector for font and colour, background images applied both to the page and to individual elements, and explicit control of image tiling through the background-repeat property exercised in its repeat, no-repeat, repeat-x and repeat-y forms.
6TH WEEK	27/06/2026	Saturday	Exercise 6: styling links with pseudo-classes. The :link, :visited, :hover, :active and :focus states were each given distinct colour and text-decoration values, and the state transitions were verified interactively by following, revisiting, hovering over and keyboard-focusing an anchor element. Specificity conflicts were recorded.

Summary of the week. The sixth week opened the styling module by establishing the three application mechanisms and the cascade governing them. Exercises 1 to 3 placed identical declarations inline, internally and externally, exposing the precedence differences that follow from source location. Exercise 4 extended styling across typography, page and element background imagery, link decoration, layered positioning governed by z-index and a custom cursor declared with a fallback value. Exercise 5 made image tiling explicit through background-repeat in its repeat, no-repeat, repeat-x and repeat-y forms, and the Saturday session exercised link pseudo-class states interactively.

Week 7: CSS Pseudo-Elements, Interactive Components and Responsive Layout

Week	Date	Day	Name of the Topic / Module Completed
7TH WEEK	29/06/2026	Monday	Exercise 7, first half: styling the first letter and the first line of an element using the <code>::first-letter</code> and <code>::first-line</code> pseudo-elements, producing a drop capital and a widened leading line. Font-size, font-weight, colour and float were applied to the generated region and the effect on surrounding text recorded.
7TH WEEK	30/06/2026	Tuesday	Exercise 7, second half: insertion of generated content before and after an element using the <code>::before</code> and <code>::after</code> pseudo-elements with the <code>content</code> property, rendering a decorative label, a separator glyph and a numbered counter against each list item without additional markup in the document.
7TH WEEK	01/07/2026	Wednesday	Exercise 8: a horizontal drop-down menu inside a navigation bar. An unordered list was laid out horizontally, submenu lists were hidden by default and revealed on hover, and absolute positioning with <code>z-index</code> kept the drop-down panel above the page content without displacing the navigation bar.
7TH WEEK	02/07/2026	Thursday	Exercise 9: alert buttons in CSS. Gradient and solid fills, border-radius, box-shadow and padding were combined into button components, colour variants distinguished informational, confirmatory and destructive actions, and transition on the background-color and box-shadow properties supplied the hover and active feedback.
7TH WEEK	03/07/2026	Friday	Exercise 10: loading buttons in CSS. A keyframe animation rotating a bordered spinner element supplied the progress indicator, animation-duration and animation-timing-function governed its cadence, and the disabled state with pointer-events suppression prevented repeated submission while the simulated request remained in flight.
7TH WEEK	04/07/2026	Saturday	Responsive layout with media queries at desktop, tablet and mobile breakpoints, box-sizing, fluid percentage widths and max-width with margin auto, combined with responsive images built from picture, source media, srcset and sizes. Rendering was compared in Google Chrome, Microsoft Edge and Mozilla Firefox.

Summary of the week. The seventh week concentrated on generated content and interface components. The two halves of Exercise 7 separated the structural pseudo-elements from the generative ones, since `::first-letter` and `::first-line` produced a drop capital and a widened leading line while `::before` and `::after` injected labels, separators and counters without additional markup. Exercise 8 resolved the drop-down navigation problem without scripting, laying the bar horizontally and revealing the submenu on hover while preserving stacking order through absolute positioning and `z-index`. Exercises 9 and 10 addressed component states, and responsive images with breakpoint checks across three browser engines closed the module.

Week 8: RDBMS Fundamentals, SQL Server and Data Definition Languages

Week	Date	Day	Name of the Topic / Module Completed
8TH WEEK	06/07/2026	Monday	Rationale for relational databases: the need for computerized data processing and the advantages of an RDBMS over file-based storage. Deployment domains including banking, airlines, universities, credit-card transactions, telecommunication, finance, manufacturing and human resource management were surveyed, and the positioning of NoSQL, SQL and Web API technologies was considered.
8TH WEEK	07/07/2026	Tuesday	Installation of Microsoft SQL Server 2019 with SQL Server Management Studio, and study of the database creation rules covering uniqueness of the database name, the CREATE DATABASE permission and the allocation of the Master Data File and Log Data File. The transaction log was examined in its disaster-recovery role.
8TH WEEK	08/07/2026	Wednesday	The SQL Server type system: the numeric types BIT, TINYINT, SMALLINT, INT and BIGINT, the character types CHAR, VARCHAR and TEXT, the binary types BINARY and IMAGE, and the date and time types DATE, TIME and DATETIME, with a real-world usage matrix mapping each type to a representative column and value.
8TH WEEK	09/07/2026	Thursday	Table creation rules, uniqueness of table names, column definitions, nullability and comma separation were demonstrated with the customer and products_catalog scripts, and the five command classes DDL, DML, DQL, TCL and DCL were enumerated with examples of each.
8TH WEEK	10/07/2026	Friday	The teacher table was created with t_id NUMERIC(3,0), t_name VARCHAR(20), t_class VARCHAR(20) and t_age NUMERIC(3,1), followed by twelve single-row INSERT statements entering the initial records one at a time, and verification of the row count and of each inserted tuple through a full-table SELECT.
8TH WEEK	11/07/2026	Saturday	A multiple-row INSERT added three records in a single statement, bringing the table to fifteen rows. A loc column was then added with ALTER TABLE ADD COLUMN, and individual UPDATE statements populated loc for t_id 1 through 15, with each affected row confirmed through a filtered SELECT.

Summary of the week. The eighth week established the relational database foundation. The rationale for centralised, normalised storage was traced through the domain applications in which such systems are deployed, and the relative positioning of document, relational and interface-based access was considered. Installation of SQL Server 2019 with SQL Server Management Studio was followed by study of the Master Data File and Log Data File generated at database creation, with the transaction log examined in its disaster-recovery role. Creation rules, permissions, the type system and the five SQL command classes were then demonstrated with executed examples in the SSMS editor.

Week 9: Data Manipulation, Query Operators and Normalisation

Week	Date	Day	Name of the Topic / Module Completed
9TH WEEK	13/07/2026	Monday	A school_name column was added and set to a single value for every row through one group UPDATE, and a t_salary NUMERIC(7,2) column was added with fifteen values. Projection followed: column aliases declared with AS and by a bare space, arithmetic on t_age and t_salary, and CONCAT() joining two character columns.
9TH WEEK	14/07/2026	Tuesday	Duplicate elimination with DISTINCT and result ordering with ORDER BY in ascending and descending sequence, then row filtering with the WHERE clause using the comparison operators, BETWEEN, IN and NOT IN. Each construct was exercised against the teacher table and the result sets compared.
9TH WEEK	15/07/2026	Wednesday	The aggregate functions COUNT(*), SUM, AVG, MIN and MAX were applied to t_age and t_salary, the LIKE operator with the per-cent and underscore wildcards was demonstrated across six pattern forms, and structural inspection was performed with EXEC sp_help on the table.
9TH WEEK	16/07/2026	Thursday	ALTER TABLE ADD COLUMN and DROP COLUMN were executed, and EXEC sp_rename was used to rename a column and then the table itself. DROP, TRUNCATE and DELETE were compared across seven stated criteria: command class, what is removed, space freed, view existence, integrity constraints, undo space and relative speed.
9TH WEEK	17/07/2026	Friday	Normalisation was applied from First Normal Form through Second Normal Form to Third Normal Form, with primary key, foreign key, composite key and referential-integrity enforcement demonstrated on the working tables. Transaction control with COMMIT, ROLLBACK and SAVE TRANSACTION and access control with GRANT and REVOKE were then exercised.
9TH WEEK	18/07/2026	Saturday	The employee table with name, occupation, working_date DATE and NUMERIC working hours was created and populated, and the take-home Employee table carrying EMPNO, EMP_NAME, DEPT, SALARY, DOJ and BRANCH was loaded with six rows and its nine required queries were answered. An index was created with the execution plan inspected.

Summary of the week. The ninth week was the most script-intensive of the internship. The teacher table was extended with the loc, school_name and t_salary columns, individual and grouped updates applied, and projection, arithmetic, concatenation, DISTINCT and ordering exercised. Filtering combined comparison, BETWEEN, IN and NOT IN with the five standard aggregates, and structural inspection used sp_help. The LIKE operator was demonstrated with per-cent and underscore wildcards across six pattern forms, renaming used sp_rename, and DROP, TRUNCATE and DELETE were distinguished across seven criteria. Normalisation to Third Normal Form with enforced referential integrity and the nine take-home queries completed the module.

Week 10: Enterprise Web Applications with ASP.NET MVC

Week	Date	Day	Name of the Topic / Module Completed
10TH WEEK	20/07/2026	Monday	Exercise 1: the Model-View-Controller architecture and the principle of separation of concerns. The request pipeline was traced in order from endpoint routing through controller selection, action invocation, model binding, view rendering and response generation, and each stage was related to its component in the framework.
10TH WEEK	21/07/2026	Tuesday	Exercise 2: creation of the project and enumeration of the default folder structure comprising Controllers, Models, Views, Views/Shared, Content, Scripts and App_Data, together with Global.asax, Web.config and packages.config. The responsibility attaching to each folder was recorded for reference during the module.
10TH WEEK	22/07/2026	Wednesday	Exercise 3: an Employee model carrying DataAnnotations attributes Required, StringLength, Range, DataType and Display together with a computed NetSalary. Exercise 4: an EmployeeDbContext with Fluent API configuration through HasKey, IsRequired, HasMaxLength, HasColumnName and Ignore, with migrations creating the SQL Server database.
10TH WEEK	23/07/2026	Thursday	Exercise 5: an EmployeeController with Index, Details, Create, Edit and Delete actions returning ViewResult and RedirectToActionResult, with action selection distinguished by the HttpGet and HttpPost attributes. TempData carried a status message across the redirect and its rendering was verified in the destination view.
10TH WEEK	24/07/2026	Friday	Exercise 6: strongly typed Razor views using the @model directive, a _Layout master page, _ViewStart, _ViewImports and the _EmployeeSummary partial, with ViewBag, ViewData and TempData compared. Exercise 7: model binding, over-posting prevented with Bind or a view model, and ModelState.IsValid driving ValidationSummary and ValidationMessageFor.
10TH WEEK	25/07/2026	Saturday	Exercise 8: conventional routing declared in RouteConfig and attribute routing with route templates and an id:int constraint. Exercise 9: custom action, authorization and exception filters registered in FilterConfig, with Authorize enforcing role-based rendering. Exercise 10: an HTML Helper rendering a table and a pay-slip statement.

Summary of the week. The tenth week opened the enterprise web application module. The Model-View-Controller architecture was traced through the request pipeline, and the generated project structure was documented with the responsibility of each folder. A model carrying DataAnnotations and a computed NetSalary was configured through the Fluent API with migrations applied to a SQL Server database, and a controller exercised the five conventional actions with redirect and status-message handling. Strongly typed Razor views, over-posting prevention and model validation were then addressed before routing, the three filter types and a helper-generated pay-slip statement completed the week.

Week 11: Consolidation, Documentation, Report Submission and Viva Preparation

Week	Date	Day	Name of the Topic / Module Completed
11TH WEEK	27/07/2026	Monday	Exercise 12: the Student Registration and Pay-Slip portal integrating the C# payroll hierarchy with the SQL Server schema. The entity was exposed through a model and an EmployeeDbContext, validated with DataAnnotations and view models, and rendered through Razor views carrying a printable pay-slip stylesheet and a role-protected route.
11TH WEEK	28/07/2026	Tuesday	Regression re-run of the C# exercise set against its specifications, and of the HTML5 and CSS deliverables. Every page was compared across Google Chrome, Microsoft Edge and Mozilla Firefox, and each difference in rendering or in interactive behaviour was recorded for reconciliation.
11TH WEEK	29/07/2026	Wednesday	Regression re-run of the SQL script library and of the MVC application. Create, read, update and delete operations, validation, filtering and the pay-slip view were verified end to end against the recorded result sets, and any divergence was traced to its originating script or view before correction.
11TH WEEK	30/07/2026	Thursday	Git repository audit and commit history review against the documentation standard agreed at induction. The deliverables were prepared for attachment: class diagrams, the HTML and CSS asset inventory, the SQL creation and manipulation scripts, and a snapshot of the MVC project taken at its final verified state.
11TH WEEK	31/07/2026	Friday	Compilation and submission of the report with the attendance-cum-activity logbook covering all the working days of the internship, together with the certificate of completion. The industry mentor's assessment was recorded, and viva preparation addressed object-oriented design, cascade and specificity behaviour, and relational query formulation.

Summary of the week. The eleventh week consolidated the four module families into a verifiable body of work. The portal exercise joined the C# payroll hierarchy to the SQL Server schema behind validated models and rendered views, and the regression passes re-executed the C# exercise set, the HTML5 and CSS pages in three browser engines, the complete script library and the MVC create, read, update and delete cycle against recorded outputs. The Git audit confirmed adherence to the documentation standard agreed at induction, class diagrams and script inventories were prepared, and report compilation, submission, the mentor assessment and viva revision closed the internship.

PART B — WEEKLY OVERVIEW OF INTERNSHIP 2 (AI3)

AI/ML Internship Program (programme code AI3, Batch 1A) — eleven weekly cycles, weekday sessions from 1:00 pm to 3:00 pm

The eleven cycles that follow record the work of the AI/ML Internship Program in the order in which it was carried out, one page for each cycle, from Python fundamentals and the data-handling libraries through the mathematics of machine learning, supervised and unsupervised learning, deep learning, natural language processing, prompt engineering and retrieval-augmented generation, model deployment, MLOps and the capstone project.

Week	Focus of the weekly cycle
Week 1	Orientation and Python fundamentals
Week 2	Python control flow, functions, NumPy and Pandas
Week 3	Data handling, mathematics for machine learning, exploratory data analysis and feature engineering
Week 4	Supervised machine learning using linear regression
Week 5	Polynomial regression, classification and K-nearest neighbours
Week 6	Tree-based models, support vector machines and unsupervised learning
Week 7	Model deployment, Docker and an introduction to reinforcement learning
Week 8	MLOps, industry practices and capstone projects
Week 9	Deep learning and natural language processing
Week 10	Prompt engineering, advanced retrieval-augmented generation and Assignments 1 to 4
Week 11	Assignments 5 to 10, portfolio preparation and completion

Each cycle closes with a summary of the modules completed, the graded work submitted and the skills carried forward into the following week.

Week 1: Orientation and Python Fundamentals

Week	Date	Day	Name of the Topic / Module Completed
1ST WEEK	20/05/2026	Wednesday	Orientation and induction to the AI/ML Internship Program (AI3, Batch 1A): the programme structure, the session schedule, the assignment series and the GitHub submission process; introduction to artificial intelligence, machine learning and deep learning and their applications in industry and governance.
1ST WEEK	21/05/2026	Thursday	Setting up the development environment with Anaconda, Jupyter Notebook and Visual Studio Code; Python basics covering variables and data types — numbers, strings, lists, tuples, dictionaries and sets — and the indexing and slicing operations available on each.
1ST WEEK	22/05/2026	Friday	Python operators: arithmetic, comparison, logical, assignment, membership and identity operators, operator precedence, and the practice programs that exercised each operator class on user-supplied input.

Summary of the week. The first week introduced the AI/ML Internship Program, its schedule of daily sessions from 1:00 pm to 3:00 pm, and the process for submitting assignments through public GitHub repositories and the programme submission form. The opening session gave an overview of artificial intelligence, machine learning and deep learning and their applications in industry and governance. The development environment was then set up with Anaconda, Jupyter Notebook and VS Code, and Python programming began with data types and operators, the building blocks of every program written later in the internship.

Week 2: Python Control Flow, Functions, NumPy and Pandas

Week	Date	Day	Name of the Topic / Module Completed
2ND WEEK	25/05/2026	Monday	Conditional statements: if, if-else, the elif ladder, nested conditions and match-case; decision-making programs such as grading, tax slabs and a three-attempt login system.
2ND WEEK	26/05/2026	Tuesday	Loops: for and while loops, the range() function, break, continue and pass, and nested loops; pattern and number programs including prime numbers, factorials and the Fibonacci series.
2ND WEEK	27/05/2026	Wednesday	Functions: definition, parameters and arguments, return values, default and keyword arguments, variable-length arguments, lambda functions, recursion and variable scope.
2ND WEEK	28/05/2026	Thursday	NumPy: creating arrays, indexing and slicing, reshaping, broadcasting, vectorised operations, and the statistical and linear-algebra functions used by every later module.
2ND WEEK	29/05/2026	Friday	Pandas I: the Series and the DataFrame, reading CSV files, inspecting data with head, info and describe, selection with loc and iloc, filtering and sorting.

Summary of the week. The second week completed the Python foundations. Conditional statements and loops were practised through a graded set of problems ranging from simple number programs to logic-based tasks such as an ATM menu and a three-attempt login system. Functions introduced reusable, modular code, including default and keyword arguments, lambda functions and recursion. The week ended with NumPy for fast numerical computation on arrays and the first part of Pandas for loading and exploring tabular data, which together form the basis of all data work in machine learning.

Week 3: Data Handling, Mathematics for ML, EDA and Feature Engineering

Week	Date	Day	Name of the Topic / Module Completed
3RD WEEK	01/06/2026	Monday	Pandas II: handling missing values and duplicates, groupby and aggregation, merging and joining DataFrames, and the apply and map transformations.
3RD WEEK	02/06/2026	Tuesday	Mathematics for ML I: scalars, vectors and matrices together with vector and matrix operations; representing a data set as a feature matrix X and a target vector y .
3RD WEEK	03/06/2026	Wednesday	Mathematics for ML II: descriptive statistics, probability and conditional probability; an end-of-week mini project on the Titanic data set applying the statistics covered.
3RD WEEK	04/06/2026	Thursday	Exploratory data analysis: univariate and bivariate analysis, histograms, box plots, scatter plots and correlation heatmaps with Matplotlib and Seaborn, and outlier detection.
3RD WEEK	05/06/2026	Friday	Feature engineering: label and one-hot encoding, feature scaling with StandardScaler, binning with <code>pd.cut()</code> , date-feature extraction and the creation of new features.

Summary of the week. The third week focused on preparing data for machine learning. Data cleaning in Pandas covered missing values, duplicates, grouping and merging. The mathematics sessions explained how a data set is represented as a feature matrix and a target vector, and reviewed the statistics and probability needed to interpret data and models, ending with a Titanic mini project. Exploratory data analysis with Matplotlib and Seaborn and feature engineering techniques such as encoding, scaling and binning completed the preparation needed before building models.

Week 4: Supervised Machine Learning using Linear Regression

Week	Date	Day	Name of the Topic / Module Completed
4TH WEEK	08/06/2026	Monday	Introduction to supervised learning: the machine learning workflow, regression against classification, training and testing data, and the <code>train_test_split</code> utility.
4TH WEEK	09/06/2026	Tuesday	Simple linear regression: the equation of a line, the cost function based on mean squared error, the intuition of gradient descent, and implementation with the scikit-learn <code>LinearRegression</code> estimator.
4TH WEEK	10/06/2026	Wednesday	Multiple linear regression: several input features, interpretation of the coefficients and the intercept, and the generation of predictions on unseen records.
4TH WEEK	11/06/2026	Thursday	Evaluating regression models: MAE, MSE, RMSE and the R^2 score, together with actual-versus-predicted and residual plots used to diagnose systematic error.
4TH WEEK	12/06/2026	Friday	Regression mini projects: house price prediction and student performance prediction, and an employee salary prediction system with feature engineering and scaling.

Summary of the week. The fourth week introduced supervised machine learning through linear regression. After an overview of the machine learning workflow and the difference between regression and classification, simple and multiple linear regression were implemented with scikit-learn, and the meaning of the coefficients, the cost function and gradient descent was discussed. Regression models were evaluated with MAE, MSE, RMSE and R^2 together with actual-versus-predicted and residual plots, and the week closed with industry-style mini projects on house prices, student performance and employee salaries.

Week 5: Polynomial Regression, Classification and K-Nearest Neighbors

Week	Date	Day	Name of the Topic / Module Completed
5TH WEEK	15/06/2026	Monday	Polynomial regression: modelling non-linear relationships, the generation of powers with PolynomialFeatures, choosing the degree, and the symptoms of underfitting and overfitting.
5TH WEEK	16/06/2026	Tuesday	Logistic regression: classification problems, the sigmoid function, log-odds, the decision boundary and log loss, implemented with the scikit-learn LogisticRegression estimator.
5TH WEEK	17/06/2026	Wednesday	Model evaluation for classification: the confusion matrix, accuracy, precision, recall, the F1-score, the ROC curve and AUC, and the classification_report() summary.
5TH WEEK	18/06/2026	Thursday	K-Nearest Neighbors: distance metrics, choosing the value of K, the importance of feature scaling, and KNN applied to both classification and regression tasks.
5TH WEEK	19/06/2026	Friday	KNN practice: accuracy-versus-K plots and the search for the best value of K on the Iris and Heart Disease data sets.

Summary of the week. The fifth week extended supervised learning. Polynomial regression showed how linear models can fit curves and how too high a degree causes overfitting. The focus then moved to classification: logistic regression and the sigmoid function, and the full set of classification metrics built on the confusion matrix, including precision, recall, the F1-score and ROC-AUC. K-Nearest Neighbors introduced distance-based learning and the importance of feature scaling, and was practised by searching for the best value of K on real data sets.

Week 6: Tree-Based Models, SVM and Unsupervised Learning

Week	Date	Day	Name of the Topic / Module Completed
6TH WEEK	22/06/2026	Monday	Decision trees: entropy, information gain and Gini impurity, the CART algorithm, tree depth, pruning and the interpretation of a fitted tree.
6TH WEEK	23/06/2026	Tuesday	Random forest: bagging, random feature selection, voting and averaging across trees, and the extraction of feature importances.
6TH WEEK	24/06/2026	Wednesday	Support Vector Machines: the hyperplane, the margin and the support vectors, linear and RBF kernels, the parameters C and gamma, and a comparison of five classifiers on the Adult Census Income data set.
6TH WEEK	25/06/2026	Thursday	Unsupervised learning: K-Means clustering, within-cluster sum of squares and the elbow method; hierarchical clustering and the reading of dendrograms.
6TH WEEK	26/06/2026	Friday	Principal Component Analysis: covariance, principal components, the explained variance ratio, and two-dimensional visualisation of high-dimensional data.

Summary of the week. The sixth week covered tree-based models, support vector machines and unsupervised learning. Decision trees were studied through entropy, information gain and Gini impurity, and random forests showed how an ensemble of trees reduces variance and provides feature importance. Support vector machines with linear and RBF kernels were compared with other classifiers on the Adult Census Income data set. The week ended with unsupervised learning: K-Means and hierarchical clustering for grouping unlabelled data, and Principal Component Analysis for reducing dimensions and visualising data.

Week 7: Model Deployment, Docker and Introduction to Reinforcement Learning

Week	Date	Day	Name of the Topic / Module Completed
7TH WEEK	29/06/2026	Monday	Model deployment concepts: the machine learning life cycle, ML pipelines, and saving and loading trained models with Pickle and Joblib.
7TH WEEK	30/06/2026	Tuesday	Flask: routes, request handling, HTML templates and forms, REST APIs returning JSON, and running the Car Price Prediction web application locally.
7TH WEEK	01/07/2026	Wednesday	Deploying machine learning web applications: Git and GitHub, requirements.txt, the Procfile and Gunicorn; the Car Price Prediction System deployed live on Render.
7TH WEEK	02/07/2026	Thursday	Docker: images, containers, the Dockerfile, layer caching and port mapping; Dockerised Car Price and Student Placement Predictor applications.
7TH WEEK	03/07/2026	Friday	Introduction to reinforcement learning: the agent, the environment, state, action, reward and policy; Gymnasium and Stable-Baselines3; a PPO agent for CartPole-v1.

Summary of the week. The seventh week marked the transition from building models to delivering them. Models were saved with Pickle and Joblib and served through Flask routes, HTML forms and JSON APIs. The Car Price Prediction System, the main project of this programme, was run locally, pushed to GitHub and deployed live on Render with Gunicorn. Docker was then used to containerise the car price and student placement applications. The week ended with an introduction to reinforcement learning, in which a PPO agent was trained with Stable-Baselines3 to balance the CartPole.

Week 8: MLOps, Industry Practices and Capstone Projects

Week	Date	Day	Name of the Topic / Module Completed
8TH WEEK	06/07/2026	Monday	MLOps: experiment tracking with MLflow, continuous integration and deployment with GitHub Actions, and the pull-request and code-review workflow.
8TH WEEK	07/07/2026	Tuesday	End-to-end example: training an Iris classifier, serving it through a Flask API with a web interface, containerising it with Docker and deploying it on Render.
8TH WEEK	08/07/2026	Wednesday	Reinforcement learning project: analysis of the LunarLander-v3 environment with its eight-dimensional observations and four discrete actions, PPO training for 500,000 timesteps, evaluation and video recording.
8TH WEEK	09/07/2026	Thursday	Recommendation systems: content-based and collaborative filtering, TF-IDF and cosine similarity, and SVD applied to the MovieLens data; a Movie Recommendation web application.
8TH WEEK	10/07/2026	Friday	Retrieval-Augmented Generation I: chatbots, document loading, chunking, embeddings, vector databases, the retriever and the language model; a document question-answering chatbot built with Google Gemini.

Summary of the week. The eighth week covered MLOps, industry practices and capstone-style projects. MLflow experiment tracking and GitHub Actions for continuous integration showed how teams test and ship machine learning code. An end-to-end Iris classifier was trained, served through a Flask API and web page, and containerised with Docker. A PPO agent was trained for 500,000 timesteps to land a spacecraft in LunarLander-v3. Recommendation systems were implemented as a movie recommender web application, and the first RAG session produced a document question-answering chatbot using Google Gemini.

Week 9: Deep Learning and Natural Language Processing

Week	Date	Day	Name of the Topic / Module Completed
9TH WEEK	13/07/2026	Monday	Artificial neural networks: the perceptron, layers, activation functions including ReLU, sigmoid and softmax, forward and backward propagation, loss functions and optimisers.
9TH WEEK	14/07/2026	Tuesday	Building artificial neural networks with TensorFlow and Keras: the Sequential model, compile and fit, epochs, batch size and validation split; an MNIST handwritten-digit classifier.
9TH WEEK	15/07/2026	Wednesday	Convolutional neural networks: convolution, kernels, stride and padding, feature maps, max and average pooling, and the architecture of a convolutional classifier.
9TH WEEK	16/07/2026	Thursday	CNN implementation: CIFAR-10 image classification with data augmentation, batch normalisation, dropout and learning-rate scheduling.
9TH WEEK	17/07/2026	Friday	Natural language processing: tokenisation, stop-word removal, stemming and lemmatisation, bag-of-words and TF-IDF representations, sentiment analysis and the basics of chatbots.

Summary of the week. The ninth week was devoted to deep learning and natural language processing. Artificial neural networks were studied from the perceptron to backpropagation and optimisers, and implemented in TensorFlow and Keras on the MNIST handwritten digits. Convolutional neural networks introduced convolution, pooling and feature maps, and were applied to CIFAR-10 with data augmentation, batch normalisation and dropout. The week ended with NLP fundamentals, from text preprocessing and TF-IDF to sentiment analysis and the basics of chatbots.

Week 10: Prompt Engineering, Advanced RAG and Assignments 1–4

Week	Date	Day	Name of the Topic / Module Completed
10TH WEEK	20/07/2026	Monday	Prompt engineering: zero-shot, few-shot and role prompting, step-by-step reasoning and structured outputs; Assignment 1, Insurance Cost Prediction with multiple linear regression, submitted.
10TH WEEK	21/07/2026	Tuesday	RAG II: answering from multiple documents, conversation memory and source citations; Assignment 2, Customer Churn Prediction with logistic regression, submitted.
10TH WEEK	22/07/2026	Wednesday	RAG III: advanced techniques including history-aware retrieval and query rewriting, tuning of chunk size, overlap and top-k, and grounding prompts to reduce hallucination.
10TH WEEK	23/07/2026	Thursday	RAG IV: production-level project structure with separate indexing and querying modules; Assignment 3, Salary Prediction with polynomial regression, and Assignment 4, Breast Cancer Classification with KNN, submitted.
10TH WEEK	24/07/2026	Friday	Deep learning practice projects: Brain Tumour MRI classification into four classes and LFW face recognition into seven identities, both with CNNs, early stopping and learning-rate reduction.

Summary of the week. The tenth week combined advanced generative-AI topics with the graded assignment series. Prompt engineering techniques were followed by three further RAG sessions covering multiple documents, conversation memory, source citations, history-aware retrieval and a production-level project structure. Deep learning practice continued with CNN projects on brain tumour MRI scans and face recognition. In parallel, the first four graded assignments, on multiple linear regression, logistic regression, polynomial regression and KNN, were submitted.

Week 11: Assignments 5–10, Portfolio and Completion

Week	Date	Day	Name of the Topic / Module Completed
11TH WEEK	27/07/2026	Monday	Assignment 5, Employee Attrition Prediction using decision tree and random forest, and Assignment 6, Weather Condition Classification using an SVM with Open-Meteo API data, submitted.
11TH WEEK	28/07/2026	Tuesday	Assignment 7, Customer Segmentation using K-Means clustering and PCA, submitted.
11TH WEEK	29/07/2026	Wednesday	Amazon quarterly report (10-Q) RAG chatbot built with FastAPI, sentence-transformers, FAISS and Gemini; consolidation of all projects into a portfolio repository.
11TH WEEK	30/07/2026	Thursday	Assignment 8, Handwritten Digit Recognition using an ANN, and Assignment 9, Cats versus Dogs Image Classification using a CNN, submitted.
11TH WEEK	31/07/2026	Friday	Assignment 10, End-to-End Heart Disease Prediction with model training, a Flask REST API and deployment on Render; completion of the internship and closing review with the instructor.

Summary of the week. The final week completed the remaining six graded assignments: a decision tree and random forest comparison, SVM classification on live weather data, customer segmentation with K-Means and PCA, digit recognition with an ANN, image classification with a CNN and, finally, an end-to-end heart disease prediction REST API deployed on Render. An Amazon quarterly-report RAG chatbot was also built with FastAPI and FAISS, and all projects were consolidated into a single portfolio repository. The internship concluded on 31st July, 2026 with the closing review, and the certificate of completion was issued by MPOne Limited.

CHAPTER 1: INTRODUCTION

1.1 The Domain: Software Engineering and Applied Machine Learning in Enterprise Systems

Enterprise information systems are built from a small number of durable ideas. State is held in a data store whose structure determines what questions may be asked cheaply; behaviour is expressed as code arranged into units whose boundaries determine how easily the system may be changed; and the value of the system to its users is delivered through an interface whose clarity determines whether the system is actually used. Contemporary practice organises these ideas into layered architectures, most commonly according to the Model-View-Controller pattern, and the maintainability of such a system depends far less on the sophistication of the algorithms it contains than on the discipline with which its units are named, its state is encapsulated, its schema is normalised and its presentation layer is separated from its logic.

Two further developments have changed what enterprise systems are expected to do. The first is scale: an e-governance platform serves millions of transactions, so query formulation, indexing and normalisation become operational requirements rather than academic exercises. The second is the availability of large quantities of structured and unstructured data, which has made machine learning a practical component of enterprise software. Forecasting service demand, detecting anomalous transactions, classifying requests and answering citizen queries from documents are now ordinary engineering tasks, and they bring a discipline of their own: the correctness of a machine learning system cannot be established by reading the code, because the behaviour is learned from data, and evaluation design therefore carries the burden that unit testing carries in ordinary software.

1.2 About the Host Organisation: MPOnline Limited

MPOnline Limited is the official e-governance service delivery organisation of the Government of Madhya Pradesh. It was incorporated in 2006 as a joint venture between the Government of Madhya Pradesh, acting through the Madhya Pradesh State Electronics Development Corporation, and Tata Consultancy Services, and it is based in Bhopal. The company develops and operates the citizen services portal hosted at www.mponline.gov.in, guided by the idea of moving from “citizen in line” to “citizen online”, and it builds and maintains the software platforms, databases and web

applications on which those services depend. Through that portal it delivers Government-to-Citizen, Government-to-Business, Business-to-Citizen and Government-to-Government services, including applications and payments, examination and recruitment processes, counselling, licences, certificates and welfare schemes, while a statewide network of authorised kiosks extends the same services to citizens who do not transact online.

The relevance of the host organisation to the work described in this report is direct. An organisation that processes applications, examination records and payments for the whole state depends on maintainable software and on correctly designed relational structures, which is the subject of Chapter 2, and it accumulates exactly the kind of structured data to which machine learning is applicable, which is the subject of Chapter 3. The organisation is ISO certified and appraised at CMMI Maturity Level 3.

1.3 The Two Internship Programmes: Scope, Structure and Responsibilities

The **Advanced Software Engineering and Development Internship** (programme code SE3) was carried out under the industry supervision of Mr. Rupesh Phalne and ran from 20th May, 2026 to 31st July, 2026, a period of seventy-three (73) calendar days, of which sixty-three (63) were working days spread over eleven weekly cycles from Monday to Saturday. The programme was structured as a supervised apprenticeship in which each working day paired a theory briefing with supervised laboratory implementation, and the role responsibility was that of an implementation engineer: to reproduce every prescribed exercise, to diagnose and correct the errors that arose, to verify each artefact against its specification, and to maintain the source under version control with documentation sufficient for another engineer to reproduce the work.

The **AI/ML Internship Program** (programme code AI3, Batch 1A) was conducted by Mr. Nishant Shrivastava over the same period, with scheduled weekday sessions from 1:00 pm to 3:00 pm. Each module combined a concept session supported by detailed notes, guided implementation in Python with Jupyter Notebook and Visual Studio Code, and practice work ranging from short exercises to complete applications. Assessment was continuous and practical: each of the ten graded assignments was submitted before a fixed deadline as a public GitHub repository documented with a README stating the objective, the data set, the method and the results. The role

responsibility was that of a data practitioner: to prepare data honestly, to choose models and metrics suited to the problem, and to deliver working services rather than notebooks alone.

Each programme exceeded the forty-five days of internship that the Department requires for evaluation, and each produced a distinct body of verifiable technical work.

1.4 Tools, Frameworks and Technology Stack

Table 1.1 lists the principal tools, languages and platforms employed across the two programmes. All are open source or available on a free tier, so the body of work reported here was built, executed and deployed without special hardware or paid infrastructure.

Table 1.1: Tools, frameworks and technologies used during the two internships

Area	Track	Tools, languages and platforms
Object-oriented programming	SE3	C# with the .NET SDK, Visual Studio 2022, the .NET console application model and MSBuild
Presentation layer	SE3	HTML5, CSS3, semantic markup, the constraint-validation API, media and canvas elements; Chrome, Edge and Firefox for cross-browser verification
Database	SE3	Microsoft SQL Server 2019, SQL Server Management Studio, T-SQL (DDL, DML, DQL, TCL and DCL), execution plans
Web application	SE3	ASP.NET MVC, Razor views, Entity Framework DbContext with the Fluent API, DataAnnotations validation, routing, filters and HTML helpers
Data handling, machine learning and deep learning	AI3	Python 3.x, Anaconda, Jupyter Notebook, VS Code, NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, TensorFlow and Keras, Gymnasium with Box2D, Stable-Baselines3, PyTorch
NLP and generative AI	AI3	LangChain, ChromaDB, FAISS, sentence-transformers, the Google Gemini API, Streamlit
Deployment, MLOps and common tools	AI3 and both tracks	Flask, FastAPI, Gunicorn, Uvicorn, Docker, Git and GitHub, Render, MLflow, GitHub Actions; Markdown documentation and a fixed delivery calendar

1.5 Organisation of the Report

The remainder of this report is arranged as follows. Chapter 2 discusses the first internship, mapping each stated objective to the activity that achieved it, describing the technical learning of each module family, recording the principal deliverables with their verified results, and closing with the skills acquired and the challenges encountered. Chapter 3 discusses the second internship in the same order, with particular attention to

its main project, a Car Price Prediction System deployed as a web service, and to the ten graded assignments. Chapter 4 states the conclusions and Chapter 5 lists the references. The weekly overview for both programmes is placed in the front matter.

CHAPTER 2: INTERNSHIP 1 DISCUSSION — ADVANCED SOFTWARE ENGINEERING AND DEVELOPMENT

This chapter discusses the first internship. It maps each original objective to the activity that achieved it and to the evidence produced, describes the technical learning of each module family in the order in which it was taught, records the principal deliverables with their verified results, and closes with the observations drawn from the work, the skills acquired, and the challenges encountered together with the manner of their resolution.

2.1 How the Internship 1 Objectives Were Achieved

The objectives stated in the front matter were achieved through supervised implementation, verification against specification, and re-execution of every artefact at the close of the programme. Table 2.1 maps each objective to the activities that achieved it and to the evidence produced.

Table 2.1: Mapping of the Internship 1 objectives to activities and evidence

Objective	How it was achieved	Evidence produced
1. C# language fundamentals	Language sessions and the introductory console exercise set in Weeks 1 and 2	Twelve console programs covering input, conversion, operators, branching and iteration
2. Class design and encapsulation	Week 3 laboratory work on the Book, building, Result and rectangle classes	Four class implementations with private state, public methods and constructor initialisation
3. Inheritance and constructor chaining	Week 3 session on base and derived declarations and the base keyword	Food, Fruit and Apple hierarchy; Clock and derived display override
4. Abstract classes and interfaces	Week 4 sessions on abstract members and interface contracts	Restaurant and McDonalds abstract implementation; RegisterForExams implemented by Student and Employee
5. Payroll hierarchy	Exercises 1 and 13 of the C# sheet	Employee extended by four staff classes with DA, HRA, PF and Staff Club Fund
6. Polymorphic weekly payroll	Exercise 11, implemented and verified against hand calculation	SalariedEmployee, HourlyEmployee, CommissionEmployee and BaseSalariedCommissionEmployee
7. Accessible HTML5 competency	Week 5 exercises on structure, forms, tables, media and accessibility	Semantic document, labelled employee form, timetable table, media and canvas page, skip link and ARIA roles
8. Style application and the cascade	Week 6 exercises on inline, internal and external styling	Three style-application demonstrations with recorded precedence outcomes

Objective	How it was achieved	Evidence produced
9. Typography, backgrounds and layering	Exercises 4 and 5 of the CSS sheet	Typography page, page and element background images, z-index layering, custom cursor
10. Pseudo-classes and pseudo-elements; components	Week 7 exercises on link states, generated content, drop-down menus and buttons	Interactive navigation bar, alert buttons and loading buttons
11. Responsible layout and cross-browser behaviour	Week 7 media-query work with rendering compared in three browsers	Breakpoint-verified pages with responsive images and a documented difference list
12. Relational database design and querying	Weeks 8 and 9 script library on SQL Server	customer, products_catalog, teacher and Employee tables with DDL, DML and DQL scripts and verified result sets
13. Normalisation and integrity	Week 9 normalisation session with key enforcement	Design progressed from First to Third Normal Form with primary, foreign and composite keys
14. ASP.NET MVC application construction	Week 10 and Week 11 exercises	Employee model, EmployeeDbContext, EmployeeController, Razor views, routing, filters and helpers
15. Professional practice	Continuous across the programme	Git repositories with commit history, README documentation and a maintained daily logbook

2.2 Technical Learning During the Internship

The programme advanced through four module families. Each was taught as a sequence of exercises in which a new construct was introduced, applied to a problem drawn from an employment context, and then verified. The subsections below record what was learned in each family and what the verification established.

2.2.1 Module 1: C# and the .NET Object-Oriented Programming Model

The first module began with the foundations of the language: value types and their conversion behaviour, the arithmetic, relational and logical operators together with their precedence, and the branching and iteration constructs. These were applied through the introductory console set, which required printing a greeting and a name on separate lines, summing and swapping numbers, multiplying three user-entered values, offering a four-operation calculator through a switch statement, generating a multiplication table, averaging four inputs, classifying an age, testing two integers for equality, determining the sign of a number, and identifying leap years through the divisibility rules for 4, 100 and 400. The exercise that carried the most insight was the palindrome check on the sample input MADAM, because the same result could be reached by

reversing the character sequence into a temporary buffer, by using `StringBuilder`, or by comparing characters inward from both ends; measuring the three implementations exposed the difference between an algorithm and its implementation.

The module then moved to class design. The `Book` class introduced private fields exposed through the methods `get()`, `show()`, `set()` and `getPrice()`; the `building` class introduced computed output through `areaperperson()`; the `Result` class held an `int` array as instance state and exposed `total()` and `avg()` over it; and the `rectangle` class demonstrated defensive validation, rejecting any dimension that was not a floating-point value greater than 0.0 and less than 20.0. Together these established the conventions that governed the remainder of the C# work: state is private, construction is explicit, and invalid input is refused at the point of assignment rather than tolerated until it produces an incorrect result somewhere else.

Inheritance and polymorphism followed. The `Food`, `Fruit` and `Apple` hierarchy established declaration, constructor chaining and the `base` keyword; the `Clock` class demonstrated overriding by rendering the same instant in 24-hour and AM/PM forms; and the `Restaurant` abstract class with its `McDonalds` implementation, together with the `RegisterForExams` interface implemented by `Student` and `Employee`, established the difference between inheriting an implementation and contracting to satisfy an interface. The distinguishing observation was that abstract classes carry behaviour to be shared and completed, whereas interfaces carry obligations to be met, and a method that accepts an interface-typed parameter can impose those obligations on every implementation without knowing anything else about it.

2.2.2 Module 2: HTML5 and CSS Presentation-Layer Engineering

The second module covered the presentation layer in two stages. The first was document structure: the semantic elements `header`, `nav`, `main`, `article`, `section`, `aside` and `footer`, together with the document type declaration, character encoding and viewport meta element. The point established early was that semantics have consequences beyond readability, because the document outline that a browser or an assistive technology derives from the markup is determined by the elements chosen rather than by the styling applied to them. The employee record form then exercised the text, date, number, select, radio, checkbox and file input types, with every control associated with a label, related controls grouped into fieldsets with legends, and client-side constraints

declared with the required, minlength, maxlength, pattern, min, max and step attributes. A class timetable demonstrated tabular markup with a caption, header cells carrying scope attributes, footer totals, and colspan and rowspan to merge the cells of a laboratory block. A media page combined an audio element with two source children, a video element with a poster and a caption track, an iframe, an inline SVG diagram and a canvas drawn through the 2D rendering context. The module closed with accessibility: a skip-to-content link as the first focusable element, alternative text on every image, aria-label on icon-only controls, a logical tab order, the navigation, main and contentinfo roles, and an aria-live region for content that changes without navigation. Constraint validation through checkValidity, reportValidity and setCustomValidity completed the picture, together with the template, details, summary and dialog elements used to build a single-page catalogue.

The second stage was styling. The three application mechanisms — inline through the style attribute, internal through a style element, and external through a linked stylesheet — were demonstrated with identical declarations so that the cascade could be observed rather than asserted, and the specificity rules governing the outcome were recorded. Typography was then controlled through the font-family, font-size, font-style, font-weight, font-variant and text-decoration properties; background imagery was applied both to the page and to individual elements with control of position, size, attachment and repetition in the repeat, no-repeat, repeat-x and repeat-y forms; link states were styled through the :link, :visited, :hover, :active and :focus pseudo-classes and verified interactively; and layering was demonstrated with position and z-index. The ::first-letter and ::first-line pseudo-elements produced a drop capital and a widened leading line, while ::before and ::after injected labels, separators and counters without altering the document. These generative techniques made possible two interface components without scripting: a horizontal drop-down menu whose submenu is revealed on hover while remaining above the page content through absolute positioning and z-index, and alert and loading buttons whose states are expressed through colour variants, transitions, a keyframe-animated spinner and the disabled state. The module closed with responsive design through media queries at desktop, tablet and mobile breakpoints, fluid widths and responsive images, and with a rendering comparison across three browser engines.

2.2.3 Module 3: Relational Database Design, Querying and Normalisation on SQL Server

The third module began with the rationale for relational storage rather than file-based records, and with the deployment domains in which such systems operate, including banking, airlines, universities, credit-card transactions, telecommunication, finance, manufacturing and human resource management. Installation of SQL Server 2019 with SQL Server Management Studio was followed by study of the database creation rules and of the two files created with every database: the Master Data File that holds tables, rows, columns and indexes, and the Log Data File that records every modification and permits recovery to a consistent state after a failure. The type system was surveyed and applied: the numeric types BIT, TINYINT, SMALLINT, INT and BIGINT, the character types CHAR, VARCHAR and TEXT, the binary types BINARY and IMAGE, and the date and time types DATE, TIME and DATETIME. The practical lesson was that type selection is a design decision with consequences for integrity, storage and index performance, and not a matter of habit.

The script library then exercised the five command classes on the teacher table and on the employee take-home table. Data definition used CREATE TABLE with explicit types and precision — t_id as NUMERIC(3,0), t_age as NUMERIC(3,1), t_salary as NUMERIC(7,2) — followed by ALTER TABLE ADD COLUMN, ALTER COLUMN, DROP COLUMN and renaming through sp_rename, and structural inspection through sp_help or the ALT+F1 shortcut. Data manipulation used single-row and multi-row INSERT statements, individual UPDATE statements and a group UPDATE that set one column to a single value across all fifteen rows, together with DELETE. Data query used projection with column aliases declared both with AS and with a bare space, arithmetic on numeric columns, concatenation with CONCAT, duplicate elimination with DISTINCT, ordering with ORDER BY, filtering with the comparison operators, BETWEEN, IN and NOT IN, pattern matching with LIKE across six wildcard forms, and the aggregate functions COUNT, SUM, AVG, MIN and MAX. Transaction control was demonstrated with COMMIT, ROLLBACK and SAVE TRANSACTION, and access control with GRANT and REVOKE.

The most consequential part of the module was normalisation. The single working table was progressively decomposed from First Normal Form through Second Normal Form to Third Normal Form, with primary, foreign and composite keys enforcing the

relationships and referential integrity preventing the insertion of a child row without its parent. The anomalies that normalisation removes were demonstrated rather than described: an update anomaly, in which a repeated attribute must be changed in several rows and may be changed in only some of them; an insertion anomaly, in which a fact cannot be recorded because an unrelated attribute is unknown; and a deletion anomaly, in which removing a row destroys the only record of another fact. The trade-off was made equally explicit, since every decomposition adds a join to the queries that reconstruct the original view. The module closed with index creation and execution-plan inspection, which showed how an access path is chosen and why an index that is not used by the plan provides no benefit.

2.2.4 Module 4: Enterprise Web Applications with ASP.NET MVC

The final module introduced the layered web application. The Model-View-Controller pattern was explained as a separation of concerns in which the Model holds state and its validation rules, the View renders that state, and the Controller mediates between them, and the request pipeline was traced in order from endpoint routing through controller selection, action invocation, model binding, view rendering and response generation. The project structure generated for an MVC application was enumerated with the responsibility attaching to each folder — Controllers, Models, Views, Views/Shared, Content, Scripts and App_Data, together with the Global.asax, Web.config and packages.config files — and the convention by which a Views subfolder mirrors the name of its controller was established.

The Employee model was created with the properties EmpId, EmpName, Department, BasicPay and DateOfJoining together with a computed NetSalary property, decorated with the Required, StringLength, Range, DataType and Display DataAnnotations attributes, and the manner in which each attribute participates in server-side validation was examined. Persistence was provided by an EmployeeDbContext deriving from DbContext, with the entity configured through the Fluent API using HasKey, IsRequired, HasMaxLength, HasColumnName and Ignore, and the database created through migrations with the payroll columns stored as NUMERIC(7,2). An EmployeeController then exposed the actions Index, Details, Create, Edit and Delete, returning ViewResult for rendering and RedirectToActionResult after a successful write, with action selection controlled by the HttpGet and HttpPost attributes and a status message carried across the redirect in TempData. Strongly typed Razor views

used the @model directive, a shared _Layout master page, _ViewStart, _ViewImports and a partial view, and the differences between ViewBag, ViewData and TempData were examined by using each to carry the same value. Model binding, the risk of over-posting when binding directly to an entity, and the defences available through the Bind attribute or a dedicated view model were then demonstrated, with ModelState.IsValid driving ValidationSummary and ValidationMessageFor. The module closed with conventional routing in RouteConfig and attribute routing with an integer constraint on the identifier, with action, authorization and exception filters registered in FilterConfig, and with HTML helpers that render a table and a monthly pay-slip statement from a collection through IHtmlContent.

2.3 Principal Deliverables and Verified Results

The principal deliverable of the internship was the C# payroll hierarchy of Exercise 1. The Employee base class carries Emp_name, Emp_id, Address, Mail_id and Mobile_no, and is extended by Programmer, AssistantProfessor, AssociateProfessor and Professor, each of which adds Basic Pay and computes Dearness Allowance at ninety-seven per cent, House Rent Allowance at ten per cent, Provident Fund at twelve per cent and Staff Club Fund at one-tenth of one per cent of Basic Pay. Gross salary is the sum of Basic Pay, Dearness Allowance and House Rent Allowance; net salary is the gross salary less Provident Fund and Staff Club Fund. Table 2.2 records the component-wise computation for a representative instance of each class, verified against hand calculation.

Table 2.2: Pay-slip computation for one representative instance of each class in the Employee hierarchy (amounts in rupees)

Class	Basic Pay	DA @ 97%	HRA @ 10%	PF @ 12%	SCF @ 0.1%	Gross	Net
Programmer	45,000.00	43,650.00	4,500.00	5,400.00	45.00	93,150.00	87,705.00
AssistantProfessor	65,000.00	63,050.00	6,500.00	7,800.00	65.00	1,34,550.00	1,26,685.00
AssociateProfessor	85,000.00	82,450.00	8,500.00	10,200.00	85.00	1,75,950.00	1,65,665.00
Professor	1,10,000.00	1,06,700.00	11,000.00	13,200.00	110.00	2,27,700.00	2,14,390.00

The second deliverable was the polymorphic weekly payroll engine of Exercise 11. Invoking a single earnings method through a collection of base-typed references produced the four different payroll rules without any test of the concrete type: a salaried employee receives a fixed weekly salary; an hourly employee receives the hourly rate

for the first forty hours and one and a half times the rate thereafter; a commission employee receives a percentage of sales through CalculateCommission; and a base-salaried commission employee receives the base salary, increased for the current period by a reward of ten per cent, plus the sales percentage. Table 2.3 records the verified outcomes for representative inputs.

Table 2.3: Verified outcomes of the polymorphic weekly payroll engine (amounts in rupees)

Employee type	Input	Amount payable
SalariedEmployee	Fixed weekly salary of 20,000.00	20,000.00
HourlyEmployee	40 hours at 500.00 plus 5 hours at 750.00	23,750.00
CommissionEmployee	Sales of 2,00,000.00 at a commission rate of 6 per cent	12,000.00
BaseSalariedCommissionEmployee	Base salary of 15,000.00 rewarded by 10 per cent, plus sales of 2,00,000.00 at 4 per cent	24,500.00

The third deliverable was the SavingsAccount model of Exercises 7 and 13, which demonstrated the semantics of a static data member by holding the annual interest rate for every saver in a single storage location. With two savers holding 30,000 and 50,000 at an annual rate of five per cent, the first month produced monthly interest of 125.00 and 208.33 respectively, raising the balances to 30,125.00 and 50,208.33; invoking the static modifyInterest method then changed the rate for both objects simultaneously, which established that the rate is not part of the state of any individual saver. The fourth deliverable was the frontend asset set described in Section 2.2.2, comprising the semantic document, the accessible employee form, the timetable table, the media and canvas page, the ten CSS exercises and the interactive components. The fifth was the SQL script library with its verified result sets and its normalised schema, and the sixth was the ASP.NET MVC pay-slip portal, which joined the C# payroll model to the SQL Server schema behind validated models and strongly typed views with a printable pay-slip stylesheet and a role-protected route.

2.4 Results and Observations

Five observations were drawn from the work of this programme. The first concerns the location of a rule. The payroll arithmetic is determined by the employment rules of the organisation rather than by the staff category, so the formula belongs in one place; placing it in the base class and allowing the derived classes only to supply Basic Pay removed duplication and made a change to the rule a single edit. The second concerns the difference between a static member and an instance member. The interest rate of the savings model is shared by every saver, so one storage location is not merely an

optimisation but the correct model of the domain, and the static method that modifies it is the correct place to enforce a change of policy. The third concerns the behaviour of overriding: invoking a salary computation through an Employee-typed reference dispatched to the derived implementation without any conditional test in the caller, which is the reason polymorphism reduces the cost of adding a new staff category.

The fourth observation concerns the presentation layer. Measuring the same declaration placed inline, internally and externally showed that precedence is decided by the cascade and by specificity rather than by proximity in the page, and that a style expressed as a pseudo-class moves a state out of the markup and into the stylesheet, where it can be changed in one place. The remaining observation concerns the database. The teacher table, with its repeated class names and its salary values held beside names and locations, was adequate for fifteen rows and inadequate for fifteen thousand: the anomalies demonstrated on it were not hypothetical, and the decomposition to Third Normal Form, with the keys that enforce it and the joins that reconstruct the original view, was the correct response. Inspection of the execution plan then showed that the value of an index depends on the queries that are actually issued.

2.5 Skills Acquired During the Internship

2.5.1 Scientific and Technical Skills

- **Object-oriented design in C#:** encapsulation of state behind access modifiers, constructor design, static members, method overloading, inheritance with constructor chaining, method overriding, abstract classes, interfaces and polymorphic dispatch.
- **Frontend engineering:** semantic HTML5 structure, accessible forms and tables, media, vector and canvas elements, ARIA roles and constraint validation; CSS application mechanisms, the cascade and specificity, selector-based typography, backgrounds, layering, pseudo-classes, pseudo-elements, keyframe animation, responsive layout and cross-browser verification.
- **Database engineering:** type selection, database and table creation rules, the DDL, DML, DQL, TCL and DCL command classes, projection, filtering, aggregation, pattern matching, schema evolution, normalisation from First to Third Normal Form, key and referential-integrity enforcement, indexing and execution-plan inspection.

- **Web application engineering:** the MVC separation of concerns, the request pipeline, DataAnnotations validation, an Entity Framework DbContext configured through the Fluent API, CRUD controllers, strongly typed Razor views, layouts and partial views, the differences between ViewBag, ViewData and TempData, model binding, over-posting defence, conventional and attribute routing, action, authorization and exception filters, and HTML helpers.
- **Professional tooling:** version control with Git, repository conventions, documentation in Markdown, and verification of every artefact against its specification.

2.5.2 Professional Skills

- **Working to a written specification:** each exercise states exact member names, formulas and behaviours, so the habit established was to read the specification before writing code and to check every stated requirement afterwards.
- **Defect diagnosis:** compilation errors arising from type mismatch, omitted conversions and unresolved names were diagnosed from the error text and the types involved rather than worked around.
- **Verification against hand calculation:** the payroll results were recomputed by hand before being accepted, which is how the difference between a plausible and a correct figure was established.
- **Documentation discipline:** every artefact was accompanied by a README stating its objective, its inputs, its outputs and how to run it, and the daily logbook was maintained throughout the programme.
- **Time management:** the eleven weekly cycles imposed a fixed calendar, and completion of each module before the next began required the exercises to be finished within their week.
- **Cross-browser verification:** rendering differences were treated as defects to be understood rather than as noise to be ignored.
- **Independent problem solving:** installation, configuration and toolchain problems were resolved from official documentation and error output.

The competencies acquired across the eleven weekly cycles are summarised in Table 2.4, in which each entry records the level reached and the work on which that assessment rests.

Table 2.4: Competency matrix at the close of Internship 1 (1 = introductory, 2 = working, 3 = independent)

Competency	Level	Basis
C# object-oriented programming	3	All sixteen exercises of the sheet implemented and verified, including the payroll hierarchy and the polymorphic payroll engine
HTML5 and accessibility	3	Semantic, form, tabular, media and accessibility exercises completed with a documented outline
CSS and responsive design	3	Ten exercises completed, including two script-free interactive components and breakpoint verification
SQL Server and T-SQL	3	Script library executed on four tables with normalisation, keys, indexes and plan inspection
ASP.NET MVC	2	The complete CRUD application was built and executed; advanced filter and helper work remained supervised
Git and documentation	3	Repositories maintained with consistent commit history and README documentation

2.6 Challenges Encountered and How They Were Resolved

The difficulties encountered during the programme were predominantly design difficulties rather than syntax difficulties: each arose from a plausible first attempt that produced a working but incorrect or unmaintainable result. Table 2.5 records the principal cases, their causes and their resolution.

Table 2.5: Challenges encountered during Internship 1 and their resolution

Challenge	Cause	How it was resolved
Payroll arithmetic duplicated in every derived class	The formula was first written in each staff class, so a change of rate had to be applied in four places	The computation was centralised in the base class and the derived classes were reduced to supplying Basic Pay, which removed the duplication
Static member treated as instance state	The annual interest rate was first assigned per object, which made a change of policy require a loop over all savers	The rate was held in a static field, changed through the static <code>modifyInterest</code> method, and its single shared storage location was demonstrated on two savers
Overridden method apparently ignored	The computation was invoked on a variable typed as the base class while the derived class was expected to take effect	Dispatch through the base-typed reference was examined, and the override signature together with the virtual modifier was corrected

Challenge	Cause	How it was resolved
Specificity conflict between style sources	Identical declarations placed inline, internally and externally produced an outcome that the source order alone did not explain	The cascade and specificity were computed for each rule, and the applicable rule was recorded rather than guessed
Percentage arithmetic on payroll columns	Numeric division and rounding on NUMERIC(7,2) columns produced a paisa-level difference from the hand calculation	The column precision was fixed, intermediate values were kept at full precision, and the final rounding point was moved to presentation
Accidental data loss through DROP	DROP, TRUNCATE and DELETE were initially treated as interchangeable ways of emptying a table	The three were separated across seven criteria and the distinction between removing a definition and removing rows was enforced in the script library
Over-posting through model binding	Binding the request directly to the entity allowed fields that the form did not expose to be set	Binding was restricted with the Bind attribute and, where the shape of the form differed from the entity, a dedicated view model was used
Route conflicts and ambiguous actions	Two actions matched the same request, and the framework could not choose between them	Attribute routing with an integer constraint was applied and the conventional route template was ordered so that each request resolved to exactly one action

The general lesson taken from these cases was that the cost of change is the measure of a design. Each of the corrected implementations became cheaper to modify afterwards, whereas each of the original implementations had been cheaper to write once and more expensive to touch thereafter.

CHAPTER 3: INTERNSHIP 2 DISCUSSION — ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

This chapter discusses the second internship. It begins by mapping each objective to the work that achieved it, then describes the technical learning of each module family, presents the main project of the programme, summarises the ten graded assignments, and records the results and the observations drawn from them, including the corrected figures obtained when the measured results were re-examined. The chapter closes with the skills acquired and the challenges faced together with the manner of their resolution.

3.1 How the Internship 2 Objectives Were Achieved

The programme was organised as a sequence of modules in which each built on the previous one, and almost every exercise followed the same workflow: define the problem, collect and clean the data, explore it and engineer features, train and evaluate a model, and finally save and deploy it. Table 3.1 maps each objective to the activities that achieved it and to the evidence produced.

Table 3.1: Mapping of the Internship 2 objectives to activities and evidence

Objective	How it was achieved	Evidence produced
1. Data handling and analysis	NumPy, Pandas, exploratory data analysis and feature-engineering sessions in Weeks 2 and 3	Data cleaning, exploration and feature engineering applied in every assignment notebook
2. Supervised learning	Regression and classification modules in Weeks 4 to 6	Assignments 1 to 6, plus the Car Price and Student Placement projects
3. Unsupervised learning	K-Means, hierarchical clustering and PCA in Week 6	Assignment 7, customer segmentation with silhouette and explained-variance reporting
4. Deep learning	Artificial and convolutional network modules in Weeks 9 and 10	Assignments 8 and 9, plus the CIFAR-10, brain tumour MRI and face recognition projects
5. NLP, prompt engineering and RAG	Four RAG sessions and a prompt-engineering session in Weeks 8 and 10	Document knowledge-base chatbot; Amazon quarterly-report chatbot with citations
6. Recommenders and reinforcement learning	Recommendation-system and reinforcement-learning modules in Weeks 7 and 8	Movie recommendation application; PPO agents for CartPole-v1 and LunarLander-v3
7. Deployment	Pickle and Joblib, Flask, FastAPI, GitHub, Render and Docker sessions in Weeks 7 and 8	Live Car Price, Student Placement, Movie Recommendation and Heart Disease services; Dockerised applications
8. MLOps and professional practice	MLOps session in Week 8, with version control and deadline-driven submission throughout	Public GitHub repositories with README documentation; all ten

Objective	How it was achieved	Evidence produced
		graded assignments submitted

3.2 Technical Learning During the Internship

3.2.1 Python, Data Handling and the Mathematics of Machine Learning

The programme began by building fluency in Python, the language used for everything that followed. The sessions covered the built-in data types and their typical uses, the arithmetic, comparison, logical, membership and identity operators, decision-making with if, elif and match-case, iteration with for and while together with break and continue, and functions with default and keyword arguments, variable-length arguments, lambda expressions and recursion. Each topic ended with graded practice questions, from simple number programs to logic-based problems such as an ATM menu, a login system permitting three attempts and income-tax slabs. These exercises built the habit of decomposing a problem into small testable steps, which later made data-processing code easier to write and to debug.

NumPy and Pandas then supplied the data toolchain. NumPy introduced the n-dimensional array, vectorised arithmetic and broadcasting, which replace slow Python loops with array operations and form the foundation of every machine learning library used later. Pandas introduced the Series and the DataFrame: reading CSV files, selecting rows and columns with loc and iloc, filtering and sorting, handling missing values and duplicates, grouping and aggregation with groupby, merging and joining, and transformations with apply and map. The mathematics module connected these tools to machine learning theory: a data set is a matrix X of features together with a target vector y , and a model is a function that maps rows of X to predictions. Descriptive statistics, probability and conditional probability were used to describe the data before any model was built, and a Titanic mini project applied the statistics covered. Exploratory data analysis followed with univariate and bivariate analysis, histograms, box plots, scatter plots, correlation heatmaps and outlier detection, and feature engineering closed the module with label and one-hot encoding, scaling through StandardScaler, binning with `pd.cut()`, date-feature extraction and the creation of new features. The lesson carried into every later assignment was that the largest gains came from the preparation of the data rather than from the choice of algorithm.

3.2.2 Supervised Learning: Regression and Classification

Supervised learning occupied three weeks. The machine learning workflow was established first, together with the distinction between regression, which predicts a quantity, and classification, which predicts a category, and the discipline of separating training from test data with `train_test_split`. Simple linear regression introduced the equation of a line, the cost function based on mean squared error and the intuition of gradient descent; multiple linear regression extended this to several input features and required the coefficients and the intercept to be interpreted rather than merely computed. Evaluation used mean absolute error, mean squared error, root mean squared error and the R^2 score, read together with actual-versus-predicted and residual plots. Polynomial regression then showed how a linear model can fit a curve through `PolynomialFeatures`, and how the choice of degree determines whether the model underfits or overfits; the same lesson was later confirmed on a data set of ten records, where the fitted curve behaved unrealistically between the points used for training.

Classification followed. Logistic regression introduced the sigmoid function, log-odds, the decision boundary and log loss; the confusion matrix and the metrics derived from it — accuracy, precision, recall, the F1-score and the area under the ROC curve — were then examined with the explicit purpose of showing where accuracy alone misleads. K-Nearest Neighbors introduced distance-based learning, the choice of K and the importance of feature scaling, and was practised by plotting accuracy against K and searching for the best value on the Iris and Heart Disease data sets. Decision trees added entropy, information gain, Gini impurity, the CART algorithm, depth control and pruning, and random forests added bagging, random feature selection, averaging and feature importance. Support vector machines completed the family with the hyperplane, the margin, the support vectors, linear and RBF kernels and the parameters C and gamma, and were compared with four other classifiers under identical conditions on the Adult Census Income data set.

3.2.3 Unsupervised Learning and Dimensionality Reduction

Unsupervised learning answered a different question: what structure is present when no labels exist. K-Means clustering was studied through the within-cluster sum of squares and the elbow method, and hierarchical clustering through the construction and reading of dendrograms. Principal Component Analysis explained covariance, the principal

components and the explained variance ratio, and was used to project high-dimensional data into two dimensions for visualisation. The assignment built on these techniques combined the two: customer records were clustered and then projected, so that the clusters could be seen as well as measured, and the silhouette score was reported alongside the explained variance so that the quality of the result could be judged rather than merely displayed.

3.2.4 Deep Learning with TensorFlow and Keras

The deep learning module began with the perceptron and advanced through layers and activation functions — ReLU, sigmoid and softmax — to forward and backward propagation, loss functions and optimisers. Artificial neural networks were then built with the Keras Sequential API: compiling a model with a loss function and an optimiser, fitting it with a stated number of epochs and a batch size, and holding back a validation split to observe generalisation rather than only training progress. The MNIST handwritten-digit classifier established the working pattern, and the training curves it produced showed directly how a network begins to overfit: the training accuracy continues to rise while the validation loss turns upward.

Convolutional neural networks addressed images properly, with convolution, kernels, stride and padding, feature maps and pooling, and an architecture in which convolution and pooling blocks feed a classification head. The CIFAR-10 project then applied the regularisation techniques that make such networks generalise: data augmentation through rotation, shifts and horizontal flips, batch normalisation, dropout and learning-rate reduction on a plateau. The comparison between the first networks and these later ones was the clearest evidence in the programme of the effect of regularisation, and it was reinforced by the brain tumour MRI and face recognition projects, in which early stopping and learning-rate reduction were applied — with the caveat, recorded in Section 3.5, that in those two projects the validation data was drawn from the test set.

3.2.5 Natural Language Processing, Prompt Engineering and Retrieval-Augmented Generation

The natural language processing module covered tokenisation, stop-word removal, stemming and lemmatisation, bag-of-words and TF-IDF representations, and sentiment analysis, which established the classical pipeline from raw text to a numeric feature matrix. Prompt engineering then introduced the techniques that make a large language

model produce usable output: zero-shot, few-shot and role prompting, step-by-step reasoning, and the specification of a structured output format.

Retrieval-Augmented Generation was taught across four sessions and was the most substantial generative-AI component of the programme. The first session established the problem that RAG solves: a language model asked about a document it has never seen will answer confidently and may be wrong, whereas a system that first retrieves the passages most relevant to a question and then instructs the model to answer only from those passages will either answer correctly or report that the answer is not present. The pipeline was implemented from its parts: documents are loaded, split into chunks, embedded, and stored in a vector database; a retriever selects the top-ranking chunks for a question; and the language model composes the answer from them. The second session extended the system to multiple documents and added conversation memory and source citations; the third addressed history-aware retrieval and query rewriting, the tuning of chunk size, overlap and top-k, and the grounding instructions that reduce hallucination; and the fourth organised the code into a production-level structure with separate indexing and querying modules. Two complete chatbots resulted: a document knowledge-base assistant built with LangChain, a persistent ChromaDB vector store, Gemini embeddings and a Streamlit interface, and a production-style Amazon quarterly-report assistant with a sliding-window chunker, local sentence-transformer embeddings, a FAISS index and a FastAPI service returning answers with citations to the excerpts used.

3.2.6 Recommendation Systems and Reinforcement Learning

Recommendation systems were introduced through content-based and collaborative filtering, TF-IDF representation and cosine similarity, and matrix factorisation by singular value decomposition on the MovieLens data. The reinforcement learning module introduced the vocabulary of the field — agent, environment, state, action, reward and policy — and the practical machinery of Gymnasium and Stable-Baselines3. A Proximal Policy Optimization agent was trained to balance the CartPole, and a second agent was trained for 500,000 timesteps to land a spacecraft in the LunarLander-v3 environment, whose eight-dimensional observations and four discrete actions were analysed before training. Evaluation by mean reward over repeated episodes, reported with its standard deviation, was the point at which the difference between a lucky policy and a reliable one became visible.

3.2.7 Deployment and MLOps

The programme treated deployment as part of machine learning rather than as an afterthought. The machine learning life cycle and the construction of pipelines were introduced, followed by saving and loading trained models with Pickle and Joblib and serving them through Flask routes, HTML templates, forms and JSON APIs. Deployment to Render required Git and GitHub, a requirements file with pinned versions, a Procfile with a Gunicorn start command and, where required, a runtime specification fixing the Python version; the experience of a build failing because the server's Python version differed from the local one is recorded in Section 3.7. Docker was then used to remove the hosting platform from the problem: a Dockerfile based on a slim Python image fixes the interpreter, installs pinned dependencies in a cached layer, exposes the service port and starts Gunicorn bound to all interfaces, so the identical image runs on a laptop and on a container platform. The MLOps session covered experiment tracking with MLflow, continuous integration and deployment with GitHub Actions, and the pull-request and code-review workflow, and it placed the version-control discipline already in use inside the wider practice of making results reproducible.

3.3 Main Project: Car Price Prediction System

3.3.1 Problem Statement and Objectives

The main project of the programme addresses the price of a used vehicle. A private seller or a dealer who wishes to sell a used car must decide an asking price without a reliable reference, and an unrealistic price delays the sale or destroys value. The objective was therefore to build a system that estimates the resale price of a vehicle from a small number of easily available attributes, and to deliver that estimate to a user through a web application that runs on public infrastructure and can be reproduced by anyone with access to the repository. The project was deliberately specified end to end — data acquisition, preprocessing, model training, evaluation, serialisation, web serving, containerisation and deployment — because the ability to carry a model through all of those stages was the competence the programme intended to build.

3.3.2 Data Set, Preprocessing and Feature Engineering

The data set is the CarDekho used-vehicle data, containing 301 records with nine columns. The target is Selling_Price in lakh rupees. The inputs available are the present

ex-showroom price of the same model, the distance driven in kilometres, the fuel type, the seller type, the transmission type, the number of previous owners and the year of manufacture, together with the vehicle name. Inspection established that the data contains no missing values, that the selling price is strongly right-skewed with most vehicles under ten lakh rupees, that the selling price correlates strongly with the present price and weakly and negatively with age, and that only two compressed-natural-gas vehicles and eleven vehicles with previous owners are present. Two properties of the data were recorded because they affect interpretation: 101 of the 301 records are two-wheelers rather than cars, and two exact duplicate rows are present with negligible effect on the fitted model.

Preprocessing produced the seven features used by the deployed model. A new `Car_Age` feature was computed as 2025 minus the year of manufacture, since age relates more directly to depreciation than a calendar year; the vehicle name was dropped because its ninety-eight distinct values across 301 rows would produce sparse columns with little generalisable signal, and the year column was dropped because the age feature replaces it. The categorical columns were encoded with explicit mappings — fuel type as petrol 0, diesel 1 and compressed natural gas 2; seller type as dealer 0 and individual 1; transmission as manual 0 and automatic 1 — chosen so that the web form can send exactly the codes used during training. The data was then divided into a feature matrix and a target vector and split into 240 training and 61 test vehicles with a fixed random seed so that the split is reproducible. Feature scaling was not applied, because tree-based models split on thresholds and are insensitive to the scale of a feature.

3.3.3 Model Development and Evaluation

A Random Forest Regressor with one hundred trees and a fixed random seed was fitted to the training data. The choice was justified rather than assumed: the relationship between price and the inputs is not linear, the data mixes continuous and categorical features, no scaling is needed, the model tolerates the outliers present in distance driven and present price, and it reports feature importances that make its behaviour explainable. Each tree is trained on a bootstrap sample and considers a random subset of features at each split, and the forest predicts the average of its trees, which reduces the variance of a single tree while retaining its ability to model interactions such as the dependence of the effect of mileage on the original price. Table 3.2 records the

evaluation of the fitted model, and the checks that were made to establish that the result is reliable rather than merely favourable.

Table 3.2: Evaluation of the Car Price Prediction model

Measure	Value	Interpretation
R ² score on the test set	0.958	The model explains about 95.8 per cent of the variation in the selling prices of vehicles it has not seen
RMSE on the test set	0.984 lakh	Typical error, with large errors weighted more heavily — about ₹98,400
MAE on the test set	0.654 lakh	Average absolute error — about ₹65,400
R ² score on the training set	0.985	The small gap from the test score indicates limited overfitting
Linear regression baseline	0.847 R ² , 1.88 lakh RMSE	A straight-line model on the same features is clearly weaker, confirming that the relationship is non-linear
Five-fold cross-validation	0.886 to 0.959, mean 0.920	Performance across five different splits of all 301 records

Two conclusions were drawn from these figures. First, the random forest is a sound choice of model for this data, because the linear baseline leaves a large part of the variation unexplained. Second, the test-set score of 0.958 is at the favourable end of the model's range, and a mean cross-validated R² of 0.920 is the more conservative estimate of how the model would behave on new records. The largest individual errors, of about 2.6 lakh rupees, occur on the few expensive vehicles for which the training data is thin. Examination of feature importances showed that the present price dominates with an importance of 0.885, followed by vehicle age at 0.062 and distance driven at 0.033, with the remaining features contributing very little. This agrees with intuition, since resale value is anchored to the original price and then reduced by age and usage, but it also establishes the principal limitation of the model: an inaccurate present price entered by a user directly distorts the estimate.

3.3.4 Serving, Containerisation and Deployment

The trained model was serialised with Pickle to `car_price_model.pkl` and reloaded to confirm that the saved file predicts correctly before it was used by the application. The Flask application defines two routes: a GET request to the root URL renders the input form, and a POST request to the predict route reads seven form fields, converts them to numbers in the order used during training, calls the loaded model and returns the page with the estimated price. The model is loaded once when the application starts, so no time is spent reloading it for each request; the prediction is rounded to two decimal places for presentation; and the conversion is wrapped in error handling that reports an

invalid input instead of failing. The form marks every numeric field as required and uses drop-down lists whose option values are the numeric codes used in training, so the encoding cannot drift between training and serving.

Deployment followed the pipeline that the programme prescribed. The project — application code, serialised model, templates, stylesheet, requirements file, runtime specification and Procfile — was committed and pushed to a public GitHub repository. The requirements file pins the exact versions of Flask, NumPy, scikit-learn, pandas and Gunicorn used during development, so that the pickled model is loaded by the same version of scikit-learn that created it, and the runtime file fixes the Python version to that of the local environment. On the hosting platform a web service was created against the repository, dependencies were installed, the service started through Gunicorn, and a public address was assigned. On the free plan a service that receives no traffic is suspended, so the first request after a pause can take up to about a minute while the service starts again. A second version of the same application was then packaged with Docker: a Dockerfile based on a slim Python image fixes the interpreter inside the container, copies and installs the pinned dependencies first so that the layer is cached when only the application code changes, exposes the service port and starts Gunicorn bound to all interfaces, which makes the image portable across platforms without modification.

3.3.5 Limitations and Future Enhancements

The evaluation identified limitations that qualify the use of the system: 301 records, of which 101 are two-wheelers, with only two compressed-natural-gas vehicles and eleven vehicles with a previous owner, so estimates for such inputs rest on very little data; heavy dependence on a single input feature; the requirement that the user compute the vehicle age rather than enter the year of manufacture; the omission of brand and model, which influence resale value but could not be encoded usefully from the available names; no range validation on the form, so unrealistic values are still processed; and the inability of a tree-based model to extrapolate beyond the range of prices seen during training. Each limitation points to a specific enhancement: train on a larger and more recent data set with brands retained as a feature; tune the hyperparameters by grid search with cross-validation and compare gradient-boosting models; build a pipeline that performs encoding and feature creation inside the model so that the form can accept a year of manufacture; add range validation in both the form and the route; expose a

JSON endpoint for other applications; and add automated tests that run before each deployment.

3.4 The Ten Graded Assignments

The ten graded assignments assessed each major module with a practical task on a real data set. Each was implemented in a notebook, documented with a README and submitted as a public repository before a fixed deadline; the final assignment additionally required a live deployment. Table 3.3 summarises the assignments, the technique applied and the headline result.

Table 3.3: Summary of the ten graded assignments

No.	Assignment	Technique	Data set (records)	Headline result
1	Medical insurance cost prediction	Multiple linear regression	Medical Cost Personal (1,338)	R^2 0.784, MAE 4,181
2	Customer churn prediction	Logistic regression	Telco Customer Churn (7,043)	Accuracy 81.97%, F1 0.636
3	Salary prediction	Polynomial regression, degree 3	Position Salaries (10)	R^2 0.876 on a two-point test set
4	Breast cancer classification	K-Nearest Neighbors, $k = 5$	Breast Cancer Wisconsin (569)	Accuracy 95.61%, recall 90.48%
5	Employee attrition prediction	Decision tree against random forest	IBM HR Attrition (1,470)	Accuracy 76.53% against 83.33%
6	Weather condition classification	Support vector machine, RBF kernel	Open-Meteo, New Delhi (4,824)	Accuracy 98.86%, affected by leakage
7	Customer segmentation	K-Means with PCA	Mall Customers (200)	Five clusters; 59.92% variance in two components
8	Handwritten digit recognition	Artificial neural network	MNIST in CSV (60,000)	Accuracy 97.41%
9	Cats against dogs classification	Convolutional neural network	Cats and dogs images (24,998)	Accuracy 85.20%
10	Heart disease prediction service	Random forest with Flask on Render	Heart Disease (1,025)	Live REST API; 100% reported, corrected in Section 3.5

Three of the assignments produced findings that were carried into the rest of the work. Assignment 1 showed the limit of an additive model: a linear fit explains most of the variation in insurance charges but spreads widely for the highest charges, because charges rise sharply when smoking is combined with a high body-mass index, and an interaction term or a tree-based model would be the natural next step. Assignment 2 put an accuracy figure into perspective: since 73.5 per cent of the test customers did not churn, a model that always predicts that no customer churns already reaches 73.5 per cent accuracy, and 151 of the 373 churners were missed, so the decision threshold and class weighting mattered more than the nominal accuracy. Assignment 5 demonstrated

the same effect more starkly: the random forest achieved the higher accuracy of 83.33 per cent while identifying only five of the forty-seven employees who left, against eighteen identified by the less accurate decision tree, because the majority-class baseline for that split was already 84.0 per cent.

3.5 Results, Observations and Corrected Figures

Table 3.4 brings together the results of the projects and assignments discussed in this chapter. The observations that follow summarise what these results, taken together, established about building machine learning systems.

Table 3.4: Consolidated results of the Internship 2 work

Project	Technique	Measure	Result
Car price prediction (main project)	Random forest regressor	R ² and RMSE on the test set	0.958 and 0.98 lakh
Medical insurance cost (A1)	Multiple linear regression	R ² and MAE	0.784 and 4,181
Customer churn (A2)	Logistic regression	Accuracy and F1-score	81.97% and 0.636
Salary prediction (A3)	Polynomial regression	R ² on two-point test and by cross-validation	0.876 and 0.837
Breast cancer (A4)	K-Nearest Neighbors, k = 5	Accuracy and recall	95.61% and 90.48%
Employee attrition (A5)	Decision tree and random forest	Accuracy with F1-score in brackets	76.53% (0.343) and 83.33% (0.170)
Weather classification (A6)	Support vector machine, RBF kernel	Accuracy	98.86%, leakage affected
Customer segmentation (A7)	K-Means with PCA	Variance in two components and silhouette	59.92% and 0.304
Handwritten digits (A8)	Artificial neural network	Accuracy	97.41%
Cats against dogs (A9)	Convolutional neural network	Accuracy	85.20%
Heart disease service (A10)	Random forest with Flask and Render	Accuracy as reported and after deduplication	100% and 75.4%
CartPole	Proximal policy optimisation	Mean reward over 100 episodes	500.00 ± 0.00
LunarLander	Proximal policy optimisation	Mean reward over 20 episodes	170.91 ± 101.20
CIFAR-10	Convolutional neural network	Accuracy	86.04%
Brain tumour MRI	Convolutional neural network	Accuracy	91.75%
Face recognition	Convolutional neural network	Accuracy	97.67%
Adult Census Income	Five classifiers compared	Best accuracy and ROC-AUC	85.05% and 0.899

The first observation is that preparation matters more than the choice of algorithm. On the car price data the random forest clearly outperformed linear regression, because the relationships are non-linear; on the Adult Census data a well-prepared logistic

regression matched the random forest. In both cases the largest gains came from preparing the data: an engineered age feature, correct encoding and scaling, and the removal of columns that carried little generalisable signal.

The second observation is that accuracy alone can mislead. When one class is rare, a model can reach a high accuracy by ignoring it, and the honest comparison is with the majority-class baseline. The churn model's 81.97 per cent must be read against the 73.5 per cent baseline, and the attrition random forest's 83.33 per cent must be read against the 84.0 per cent baseline that it failed to beat in the terms that matter. Precision, recall, the F1-score and an explicit baseline give a truer picture than accuracy.

The third observation is that results which look too good usually point to a problem in the data. Two cases were found in this programme. The heart disease model reported perfect accuracy because most test records were duplicates of training records, and the weather model reported nearly 99 per cent accuracy because the label was derived from one of the input features. The practical consequence is a routine: check for duplicate rows, check how the target was created, and ask whether any feature could reveal the answer, before trusting a favourable score.

The fourth observation is that the design of the evaluation matters as much as the model. A test set of two points, or of sixty-one vehicles, gives a noisy estimate, and cross-validation gives a more dependable one. In the cats-against-dogs, brain tumour and face recognition projects the test set was also used during training for validation, so it was no longer an independent check; a separate validation split keeps the final test honest. The fifth observation is that neural networks overfit quickly without regularisation: in the first networks the validation loss began to rise after a few epochs while training accuracy continued to improve, and the later networks that used augmentation, dropout, batch normalisation, early stopping and learning-rate reduction generalised far better. The sixth observation is that deployment depends on reproducibility: every deployed project worked only after the Python version and the library versions on the server matched those used to train and save the model, which is what pinning versions, fixing the interpreter and packaging with Docker achieved.

Where a published figure was later found to be optimistic or imprecise, the corrected value is recorded in Table 3.5 and is the value used throughout this chapter.

Table 3.5: Corrected and cross-checked results

Project and issue	Originally reported	Corrected or cross-checked result
Heart disease service (A10): duplicate records	Accuracy 100%	75.4% on a deduplicated test set, and 82.8% with five-fold cross-validation
Salary prediction (A3): two-point test set	R^2 0.876	Cross-validation R^2 0.837 at degree 3 and 0.932 at degree 4
Breast cancer (A4): choice of decision threshold	Recall 90.48% with four missed cases	Recall 97.6% with one missed case at a threshold of 0.2, at the cost of six false alarms instead of one
Customer segmentation (A7): choice of features	Silhouette 0.304 on four features	Silhouette 0.555 using income and spending score alone
Car price (main project): single test split	Test R^2 0.958	Mean five-fold cross-validation R^2 0.920
Weather classification (A6): target leakage	Accuracy 98.86%	Not meaningful; the model must be retrained without the temperature feature

3.6 Skills Acquired During the Internship

- **Python for data science:** writing modular Python and using NumPy and Pandas to load, clean, transform and summarise real data sets.
- **Machine learning with scikit-learn:** building, tuning and comparing regression, classification and clustering models, with preprocessing steps such as encoding and scaling inside a reproducible pipeline.
- **Model evaluation and validation:** choosing metrics that match the problem, reading confusion matrices, using baselines, cross-validation and decision thresholds, and detecting leakage, duplication and overfitting.
- **Deep learning with TensorFlow and Keras:** designing artificial and convolutional networks, reading training curves, and applying augmentation, dropout, batch normalisation and early stopping.
- **Generative AI and Retrieval-Augmented Generation:** chunking and embedding documents, using vector databases, designing prompts for grounded answers, and handling API rate limits.
- **Reinforcement learning:** formulating problems as environments with states, actions and rewards, and training and evaluating PPO agents.
- **Deployment and MLOps:** serialising models, building web applications and REST APIs with Flask and FastAPI, serving them with Gunicorn, containerising with Docker, deploying on Render, and understanding experiment tracking and continuous integration.

- **Data visualisation and communication:** communicating findings through correlation heatmaps, confusion matrices, learning curves and scatter plots, and writing README documentation that states objective, method and result.
- **Professional practice:** time management under ten fixed deadlines, working to written specifications, version-control discipline with ignore files for data and secrets, keeping credentials out of repositories through environment variables, and the critical habit of re-examining any result that looks too good.

3.7 Challenges Faced and How They Were Overcome

The difficulties of this programme were practical rather than theoretical: environments, deployments, service limits and the interpretation of results. Table 3.6 records the principal cases, their causes and their resolution.

Table 3.6: Challenges faced during Internship 2 and their resolution

Challenge	Cause	How it was resolved
Build failure while deploying to the hosting platform	The platform's default Python version was newer than the local version, and some pinned libraries were unavailable for it	The Python version of the service was pinned to match the local environment and all library versions were kept pinned in the requirements file
Local port conflict	The port used by default by the web framework was already occupied by an operating-system service	The API was run on an alternative port for local testing
Installing the physics dependency for LunarLander	The Box2D extra compiles C++ code and requires a build tool that was not present	The required build tool was installed from a package channel before the dependency itself
API rate limits while embedding documents	The free tier of the language-model API limits requests per minute	Chunks were embedded in batches with pauses between batches and an automatic retry after the rate limit was exceeded
Downloading data sets inside notebooks	The data platform's API requires account credentials	Credentials were entered at run time and never stored in the notebook, and a download helper was used where available
Large deployment image for the RAG chatbot	The sentence-transformer library installs a deep-learning framework with multi-gigabyte accelerator libraries by default	The CPU-only build of the framework was installed through an additional package index recorded in the requirements file
Keeping repositories clean and secure	Data sets, environments, caches and credential files must not be committed	Ignore files were added and only the small model files needed by the deployed applications were committed
Imbalanced classes	The churn and attrition data contain few positive cases	Results were evaluated with precision, recall and the F1-score against a majority-class baseline, and class weighting and threshold tuning were identified as remedies
Overfitting of neural networks	Large dense layers and many epochs without regularisation	Augmentation, dropout, batch normalisation, early stopping and

Challenge	Cause	How it was resolved
		learning-rate reduction were applied in the later network projects
Scores that looked too good	Duplicate records in one data set and a feature derived from the target in another	Duplicates and target-derived features were checked, and the results were re-evaluated after deduplication and with cross-validation

The most valuable lesson taken from these difficulties is that reliable machine learning rests on two habits: making every environment reproducible, by pinning versions and packaging applications, and verifying every result, by checking the data, the design of the evaluation and a simple baseline before trusting a number.

CHAPTER 4: CONCLUSION

The two internship programmes at MPOne Limited, carried out concurrently from 20th May, 2026 to 31st July, 2026, together provided a complete grounding in the construction of enterprise software and in the building and delivery of machine learning systems. The Advanced Software Engineering and Development Internship covered C# and the .NET object-oriented model, HTML5 and CSS presentation-layer engineering, relational database design and querying on SQL Server, and layered web application construction with ASP.NET MVC. The AI/ML Internship Program covered Python and data analysis, the mathematics of machine learning, supervised and unsupervised learning, deep learning, natural language processing and Retrieval-Augmented Generation, recommendation systems, reinforcement learning, and deployment with Flask, FastAPI, Docker, GitHub and Render. Every objective stated at the beginning of the programmes was achieved, as evidenced by the verified deliverables described in Chapters 2 and 3 and by the ten graded assignments submitted within their deadlines.

The principal artefact of the first programme was the C# payroll hierarchy, in which an Employee base class is extended by Programmer, AssistantProfessor, AssociateProfessor and Professor, each computing Dearness Allowance at ninety-seven per cent and House Rent Allowance at ten per cent of Basic Pay and deducting Provident Fund at twelve per cent and Staff Club Fund at one-tenth of one per cent. Verified against hand calculation, it formed the domain core of the ASP.NET MVC pay-slip portal, which joined it to a normalised SQL Server schema behind validated models and strongly typed views. The principal artefact of the second programme was the Car Price Prediction System, whose Random Forest model predicts the resale price of a vehicle with an R^2 of 0.958 on unseen data and a cross-validated R^2 of about 0.920, served through a Flask application deployed on Render and packaged as a Docker image.

Three conclusions are drawn from the work. The first is that design discipline, not cleverness, determines whether software survives: centralising the payroll formula, encapsulating state, normalising the schema to Third Normal Form and separating rendering from logic each removed a class of future defect. The second is that a measurement is only as good as the design of the evaluation that produced it, since a two-point test set, a validation set drawn from test data, duplicate records shared between partitions and a feature derived from the target each produced results that

looked excellent and were not. The third is that deployment is part of the work rather than a stage after it, because a model that cannot be reproduced, packaged and reached by a user has not delivered value. Professionally, both programmes required delivery to a fixed calendar, documentation to a reviewable standard and maintenance under version control. The immediate next steps are to extend the main project with a larger data set, retained brand features, hyperparameter tuning and automated tests, and to continue studying MLOps practice so that models may be improved and released without risk to the services that depend on them.

CHAPTER 5: BIBLIOGRAPHY

5.1 Books and Research Papers

- [1] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [2] W. McKinney, Python for Data Analysis, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, An Introduction to Statistical Learning with Applications in Python. Cham, Switzerland: Springer, 2023.
- [4] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
- [5] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [6] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [9] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.
- [11] A. Raffin et al., “Stable-Baselines3: Reliable reinforcement learning implementations,” Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.
- [12] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 9459–9474.
- [13] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.
- [14] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, Art. 19, 2015.
- [15] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-based learning applied to document recognition,” Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- [16] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Technical Report, University of Toronto, 2009.
- [17] N. Srivastava et al., “Dropout: A simple way to prevent neural networks from overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.
- [18] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. 32nd International Conference on Machine Learning (ICML), 2015, pp. 448–456.

- [19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. 3rd International Conference on Learning Representations (ICLR), 2015.
- [20] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.
- [21] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [22] A. Freeman, Pro ASP.NET Core MVC, 7th ed. New York, NY, USA: Apress, 2022.

5.2 Standards, Specifications and Vendor Documentation

- [23] World Wide Web Consortium, HTML Living Standard and CSS specifications, <https://html.spec.whatwg.org> and <https://www.w3.org/Style/CSS/>.
- [24] Web Accessibility Initiative, Web Content Accessibility Guidelines (WCAG) 2.2, <https://www.w3.org/WAI/standards-guidelines/wcag/>.
- [25] Microsoft, C# language reference and .NET documentation, <https://learn.microsoft.com/dotnet/csharp/>; Entity Framework and SQL Server documentation, <https://learn.microsoft.com/ef/> and <https://learn.microsoft.com/sql/>.
- [26] scikit-learn developers, scikit-learn user guide and API reference, <https://scikit-learn.org/stable/>.
- [27] TensorFlow and Keras documentation, <https://www.tensorflow.org> and <https://keras.io>.
- [28] Flask documentation, <https://flask.palletsprojects.com>; FastAPI documentation, <https://fastapi.tiangolo.com>; Unicorn documentation, <https://unicorn.org>.
- [29] Docker documentation, <https://docs.docker.com>; Render documentation, <https://render.com/docs>.
- [30] Stable-Baselines3 documentation, <https://stable-baselines3.readthedocs.io>; Gymnasium documentation, <https://gymnasium.farama.org>.
- [31] LangChain documentation, <https://python.langchain.com>; Google Gemini API documentation, <https://ai.google.dev>; Streamlit documentation, <https://docs.streamlit.io>.
- [32] MLflow documentation, <https://mlflow.org>; GitHub Actions documentation, <https://docs.github.com/actions>.
- [33] MPOnline Limited, official citizen services portal, <https://www.mponline.gov.in>, and company website, <https://www.mponlinelimited.com>.
- [34] MPOnline Limited, Advanced Software Engineering and Development Internship and AI/ML Internship Program (AI3) lecture notes, tutorials and assignment briefs, Bhopal, May–July 2026.

5.3 Data Sets and Application Programming Interfaces

- [35] Kaggle data sets used during the programme, <https://www.kaggle.com/datasets/>: nehalbirla/vehicle-dataset-from-cardekho; mirichoi0218/insurance; blastchar/telco-customer-churn; uciml/breast-cancer-wisconsin-data; oddrationale/mnist-in-csv; johnsmith88/heart-disease-dataset.
- [36] Open-Meteo, Historical Weather API, <https://open-meteo.com>.
- [37] GroupLens Research, MovieLens Latest Datasets (small), <https://grouplens.org/datasets/movielens/>.

[38] U.S. Securities and Exchange Commission, EDGAR full-text filing system,
<https://www.sec.gov/edgar>.